

TUNABLE NON-LINEAR MANIFOLD ROMs FOR ELIPTIC, PARABOLIC, AND HYPERBOLIC PDES VIA MATRIX-FREE PETROV–GALERKIN PROJECTION

Anonymous authors

Paper under double-blind review

ABSTRACT

Neural operators such as Fourier Neural Operators and DeepONets typically deliver one (accuracy, speed) operating point per trained model; moving it generally requires retraining. We present a nonlinear-manifold reduced-order model (NM-ROM) for Poisson, heat, viscous Burgers and incompressible Navier–Stokes equations that exposes a deployment-time accuracy–speed tradeoff from one trained decoder, set by the width of an importance-ordered spatial bank (accuracy) and the solver tolerance and empirical-quadrature rule (cost). The framework combines a frozen coordinate-network spatial bank, ordered once after training, with a nonlinear head; exact Dirichlet enforcement in the bank; least-squares Petrov–Galerkin projection of the discrete residual onto fixed weak tests, with every linear operator preassembled; matrix-free JAX evaluation; and NNLS empirical quadrature for Burgers advection, validated on held-out states but not confirmed on re-draws at 4096^2 . Across Poisson and heat in two and three dimensions, viscous Burgers in two and Navier–Stokes in three, at meshes up to 4096^2 , the frozen model keeps its accuracy as the mesh is refined, so its speedup over the named solvers reaches $106\times$ at 0.74 % error against conjugate gradients on Poisson, $103\times$ at 0.49 % against Crank–Nicolson CG on a sealed held-out heat cohort, and $4.87\times$ at 0.60 % on Burgers against Newton–BiCGStab ($5.38\times$ at 1.33 % on held-out cases), and $6.67\times$ at 0.15 % on Navier–Stokes at 96^3 against a pseudo-spectral solver; at matched latent dimension its fast setting is more accurate than a reproduced shallow-masked-autoencoder NM-ROM and POD-LSPG. The named solvers remain more accurate; three-dimensional heat meets its accuracy target but beats CN–CG only at 128^3 .

1 INTRODUCTION

Neural operators for partial differential equations (PDEs) (Li et al., 2021; Lu et al., 2021; Kovachki et al., 2023) typically deliver one (accuracy, wall-clock) operating point per trained model, with no deployment-time knob to trade compute for accuracy; moving the point generally requires retraining. Full-order methods (FOMs) solved with iterative solvers such as conjugate gradients (Hestenes and Stiefel, 1952) expose this knob through their tolerance, but their cost grows with the mesh. Reduced-order models (ROMs) approach the problem from the opposite direction: linear projection-based ROMs (Benner et al., 2015; Sirovich, 1987) can require many basis functions for moving fronts (Cohen and DeVore, 2015), and nonlinear-manifold ROMs (NM-ROMs) learn a more compact representation (Lee and Carlberg, 2020; Kim et al., 2022), but their online nonlinear solve can offset the benefit of fewer unknowns. The key open question is thus: *can one trained NM-ROM give a deployment-time accuracy–cost choice, keep its accuracy as the mesh is refined, and be faster than named iterative full-order solvers while being more accurate than prior nonlinear-manifold ROMs?*

This paper answers that question largely affirmatively for two-dimensional Poisson, heat and viscous Burgers, and three-dimensional Navier–Stokes. We present an NM-ROM whose distinguishing property is a deployment-time accuracy–cost choice from a single trained model. It represents the solution by a learned spatial bank multiplied by a nonlinear map of latent variables. After training, the bank is ordered once by how much of the training data each of its functions carries, so a query can solve in the span of its first R' functions, or with the nonlinear head, without changing a learned

weight. The bank width is the accuracy control; stopping tolerances and empirical quadrature set the cost of the solve. All online coefficients come from the weak PDE residual, without access to the reference solution. The accurate error (Table 1) stays near its coarse-mesh level up to 4096^2 while the speedup over the named solver grows, and on held-out Burgers cases the model is far more accurate than a reproduced NM-ROM and POD-LSPG (Table 2). On Navier–Stokes the nonlinear head is far more accurate than a linear span of the same size. Three-dimensional heat meets its accuracy target but is faster than CN–CG only at 128^3 .

Contributions.

1. **A tunable NM-ROM with a deployment-time accuracy/speed tradeoff from a single trained model.** An importance-ordered bank provides nested deployment settings that change approximation capacity without retraining, and the nonlinear head provides an accurate end where a linear span is not enough. We measure the full ladder on development and held-out cases.
2. **Architectural choices that make this tradeoff achievable.** A coordinate-network spatial bank, a nonlinear head with a linear skip, and a one-time ordering of the bank support compact latent solves, exact boundary enforcement and node-local evaluation.
3. **A matrix-free JAX implementation that makes the framework practical.** With precomputed linear operators, matrix-free differentiation and validated empirical quadrature, query time grows more slowly with the mesh than the named solvers’: the accurate setting reaches 0.74 % on Poisson at $106\times$ the speed of CG, and 0.60 % on Burgers at $4.87\times$ that of Newton–BiCGStab, at 4096^2 in the same allocation.

2 RELATED WORK

Linear and non-linear manifold ROMs. Linear ROMs approximate solutions in a fixed subspace (Benner et al., 2015; Sirovich, 1987); moving fronts can require a large basis (Cohen and DeVore, 2015; Greif and Urban, 2019). Nonlinear-manifold ROMs combine learned representations with residual projection (Lee and Carlberg, 2020; Kim et al., 2022), including quadratic manifolds (Geelen et al., 2022; Barnett and Farhat, 2022), neural-field representations (Kim et al., 2024) and adaptive bases (Peherstorfer and Willcox, 2015; Carlberg, 2015). Least-squares Petrov–Galerkin projection (Carlberg et al., 2011) and preassembly of reduced operators (Ştefănescu and Sandu, 2014; Weder et al., 2024) are established components. We differ from these by a coordinate-network bank with an exact boundary factor, a head with a linear skip, and a bank ordered after training whose nested widths give deployment-time settings from one trained model, as runtime-tunable networks do (Yu et al., 2019; Cai et al., 2020).

Neural operators and hyper-reduction. FNO (Li et al., 2021), DeepONet (Lu et al., 2021), U-Net (Ronneberger et al., 2015; Takamoto et al., 2022) and Transolver (Wu et al., 2024) learn solution maps. Their evaluation resolution or inference procedure can also change accuracy and cost; our mechanism instead changes the trial space used to solve the PDE. Empirical quadrature (Hernández et al., 2017; Yano and Patera, 2019) reduces nonlinear residual evaluation through non-negative weights. We retain that construction, testing its error on states reached by the solver and through independent re-draws. The main iterative linear-system comparator is conjugate gradient (Hestenes and Stiefel, 1952).

3 METHODOLOGY

To answer whether one trained model can expose a deployment-time accuracy/speed tradeoff, we build the NM-ROM from three components. First, we define a trial manifold whose decoder enforces Dirichlet conditions exactly and whose spatial bank is ordered by importance, so that a query can use fewer of its functions (§3.1). Second, we project the full-order residual onto that manifold by least-squares Petrov–Galerkin projection onto fixed weak tests, which leaves a small least-squares problem at every query (§3.2). Third, we preassemble every linear term exactly and apply empirical quadrature to the nonlinear term (§3.3). The architecture in §3.4 is then designed to support these requirements in practice. Throughout, $u \in \mathbb{R}^n$ is the full-order state on a uniform grid with N intervals per axis and $A \in \mathbb{R}^{n \times n}$ the discrete negative Laplacian. Four sizes recur: R functions in the bank, of which a query uses the first R' ; k latent coordinates; and M weak tests, with $k, R' < M \ll n$. The empirical

quadrature uses m nodes. The formulas are for the scalar Dirichlet case; Appendix A gives the rest, and Figure 1 shows the model and the online solve.

3.1 TRIAL MANIFOLD WITH EXACT DIRICHLET ENFORCEMENT

We write a solution as a fixed set of spatial functions times coefficients that a small network produces. The bank $G \in \mathbb{R}^{n \times R}$ holds the R functions, one per column, sampled at the mesh nodes. The head $h_\theta : \mathbb{R}^k \rightarrow \mathbb{R}^R$ maps k latent coordinates z to the R coefficients, and the decoder is the product $\mathcal{D}(z) = G h_\theta(z)$. There is no encoder: a query finds z by solving the PDE residual (§3.2), not by encoding a field. Each column of the bank is a random-Fourier-feature coordinate network g_ϕ (Tancik et al., 2020), read at the node coordinates and multiplied by a factor that vanishes on the boundary. On the square,

$$G_{x,:} = \mu(x) g_\phi(x)^\top, \quad \mu(x) = 16x_1(1-x_1)x_2(1-x_2). \quad (1)$$

Since $\mu = 0$ on the boundary Γ , every column of G , every trial state and every derivative $\partial u / \partial z$ is zero on Γ at any resolution. Homogeneous Dirichlet data are therefore enforced by the decoder, and the Galerkin projection needs no penalty term. The cube uses the product of the same factor in three coordinates, and the L-shaped domain a factor that vanishes on its six edges.

Ordering the bank. The bank is trained as a whole, so its columns come in no particular order and a subset of them is a poor basis. After training we rotate it once so that its columns are ordered by how much of the training data they carry. Let $G = Q_G R_G$ be a thin QR factorisation. For every training field u_i we take its bank coefficients c_i and stack the scaled vectors $a_i = R_G c_i / \|u_i\|$ as the rows of a matrix $\mathcal{A}$. With the singular value decomposition $\mathcal{A} = U S V^\top$ we set

$$\hat{G} = G T, \quad T = R_G^{-1} V. \quad (2)$$

The columns of $\hat{G}$ are orthonormal and ordered by the singular values in S . This is the POD of the training fields inside the span of the bank, and it uses training data only. Because T is invertible, the full-width bank gives the same trial states as before (to 10^{-10} in our runs), and the head is kept unchanged. A query then has two ways to solve,

$$u = G h_\theta(z) \quad (\text{head, } k \text{ unknowns}), \quad u = \hat{G}_{R'} c, \quad c \in \mathbb{R}^{R'} \quad (\text{span, } R' \text{ unknowns}), \quad (3)$$

where $\hat{G}_{R'}$ holds the first R' columns. The bank width R' is the accuracy knob. The columns are nested, so every smaller width is contained in a larger one and no weight is retrained. Fewer columns give a smaller trial space and a cheaper solve, both in the iteration and in returning the field u , which costs $O(nR')$. The head reaches states off every linear span through its nonlinearity; the span reaches only its R' directions but is linear in its unknowns.

3.2 PROJECTING THE PDE ONTO THE MANIFOLD

Each PDE gives a discrete residual $r(u) \in \mathbb{R}^n$ that is zero at the discrete solution. A trial state has only $p = k$ or $p = R'$ free numbers, so r cannot be driven to zero. We ask instead that it be small against M fixed directions, the rows of the test matrix $P \in \mathbb{R}^{M \times n}$. On the Dirichlet square the tests are the low-frequency tensor-product sine vectors, which are eigenvectors of the five-point operator, $PA = \Lambda P$. Writing ξ for the unknowns (z for the head, c for the span), every query solves

$$\xi^* = \arg \min_{\xi} \frac{1}{2} \|r_w(\xi)\|_2^2, \quad r_w(\xi) = \Lambda_\star^{-1} P r(u(\xi)) \in \mathbb{R}^M, \quad M > p. \quad (4)$$

The diagonal $\Lambda_\star$ puts the rows on a common scale (Appendix A). This is least-squares Petrov–Galerkin projection with fixed tests (Carlberg et al., 2011; Lee and Carlberg, 2020), and $M > p$ makes it overdetermined. We solve it by a method suited to each PDE.

Elliptic (Poisson). The residual is $r(u) = Au - f$. With $u = Gc$, testing and rescaling give a residual that is linear in the coefficients,

$$r_w = B_0 c - b_0, \quad B_0 = PG, \quad b_0 = \Lambda^{-1} P f, \quad (5)$$

with $c = h_\theta(z)$ for the head. For the span, B_0 is replaced by its first R' rotated columns $B_0 T_{:,1:R'}$ and $c \in \mathbb{R}^{R'}$. The residual is then linear in the unknowns, so the query is one least-squares solve, by a thin QR factorisation of this fixed $M \times R'$ matrix. With the head it is nonlinear in z , and we iterate Levenberg–Marquardt in k unknowns (Appendix A.1).

Parabolic (heat). The semi-discretisation $du/dt = -\kappa Au$ is advanced by Crank–Nicolson. The manifold is substituted into the fully discrete step before projection. With the head each step solves

$$z_{n+1} = \arg \min_z \|B_0 h_\theta(z) - D B_0 h_\theta(z_n)\|_2, \quad (6)$$

where D scales each test by its Crank–Nicolson factor. In the span the same step is linear in c , so the rollout is a fixed linear recurrence (15).

Hyperbolic (Burgers). For $u_t + u(u_x + u_y) = \nu \Delta u$, with a sign-upwind stencil and backward Euler, the tested step for the coefficients c is

$$r_{w,n} = (I + \Delta t \nu \Lambda)^{-1} [B_0(c - c_n) + \Delta t (\hat{N}(c) + \nu \Lambda B_0 c)]. \quad (7)$$

Diffusion is the preassembled linear term. The advection $\hat{N}(c)$ is nonlinear in c , so both the head and the span iterate Levenberg–Marquardt at every step. The upwind stencil is in Appendix A.3.

Incompressible flow (Navier–Stokes). Periodic three-dimensional Navier–Stokes in our family carries localised vortices across the domain, and a fixed bank needs many functions to represent a moving structure (Cohen and DeVore, 2015). We therefore write the state in a frame that moves with the flow, $u(x, t) = v(x - s(t), t)$, and put the bank on v . Advection, diffusion and the Leray projector all commute with translation, so the moving-frame equation gains one term, $\dot{s} \cdot \nabla v$, which is linear in the frame increment $\delta = s^{n+1} - s^n$. The increment joins the unknowns: each step solves for (z, δ) or (c, δ) , that is $k + 3$ or $R' + 3$ numbers, from the same weak residual on fixed solenoidal Fourier tests. For this problem the bank is a rank-64 POD of centred training states, not a coordinate network (Appendix A).

Solver and exits. The iterative solves use a damped Levenberg–Marquardt step (Marquardt, 1963) that is kept only when the residual decreases; Navier–Stokes uses a fixed number of damped Gauss–Newton sweeps per step. A stall is reported as early-stopped, separately from stationarity and from a residual threshold. Every head solve starts from a stored training code (Appendix A.6).

3.3 HYPER-REDUCTION

The manifold reduces the unknowns from n to p , but the tested residual $Pr(Gc)$ still visits every mesh node, so each evaluation costs $O(nR)$ and grows with the mesh. Left in place, this cost cancels the benefit of solving for few unknowns. We remove it in two ways, one for linear terms and one for the nonlinear term.

Linear terms: preassembled and exact. A linear term is tested once, offline. For the Laplacian,

$$P(AGc - f) = Bc - b, \quad B = PAG \in \mathbb{R}^{M \times R}, \quad b = Pf, \quad (8)$$

and on the square, where $PA = \Lambda P$, $B = \Lambda PG$ requires no application of A . B is built once per mesh, and the span at width R' uses its first R' rotated columns $BT_{:,1:R'}$. Online, a linear term costs $O(MR)$, is exact and never returns to the grid. For Poisson and heat this is the whole residual, so their reduced solves make no approximation beyond the trial space. The quadratic term of Navier–Stokes in a fixed bank is also tested exactly, as a precomputed third-order tensor.

The nonlinear term: empirical quadrature. Burgers advection $N(u)$ is the term that stays nonlinear in the coefficients and is not a polynomial we can preassemble. Its tested value is a sum over the nodes, $PN(u) = \sum_{i=1}^n P_{:,i} N_i(u)$, and empirical quadrature (Hernández et al., 2017; Yano and Patera, 2019) replaces that sum by a weighted sum over a sparse node set $\mathcal{S}$,

$$PN(u) \approx \sum_{i \in \mathcal{S}} w_i P_{:,i} N_i(u), \quad w_i \geq 0, \quad |\mathcal{S}| = m \ll n. \quad (9)$$

The weights are shared by every state, so they are fitted once. For n_{fit} stored training codes $c^{(\ell)}$, the tested per-node contributions $P_{:,i} N_i(Gc^{(\ell)})$ form the columns of a matrix $\mathcal{G}$ and the exact full-grid values $PN(Gc^{(\ell)})$ form the target β . We solve the non-negative least-squares problem (Lawson and Hanson, 1974)

$$\min_{w \geq 0} \|\mathcal{G} w - \beta\|_2, \quad (10)$$

which returns a sparse w whose support is $\mathcal{S}$ (Appendix A.5). The stencil value N_i reads u at node i and its four neighbours $\mathcal{N}(i)$, so the decoder is needed only on $\mathcal{S}^+ = \mathcal{S} \cup \bigcup_{i \in \mathcal{S}} \mathcal{N}(i)$, with

$|\mathcal{S}^+| \leq 5m$. Because each bank column is a coordinate network (§3.4), those rows are computed once and cached as an $m \times 5 \times R$ block, and the rest of the mesh is not touched online.

Cost. With linear terms preassembled and advection on the rule, every online array has size $M \times R$, $m \times R$ or $M \times p$. One Levenberg–Marquardt iteration, which forms the tested residual and its Jacobian and solves the damped normal equation, costs at most

$$\mathcal{O}(MR(m+p)), \quad (11)$$

independent of the number of mesh nodes n ; dense evaluation replaces m by n . Returning the field u at an output time is one further $\mathcal{O}(nR')$ product, outside the iteration, and at our finest meshes it is the largest part of a linear query. This is why the bank width is also a cost knob. The iterative full-order solvers sweep all n nodes in every sparse matrix–vector product, and for CG on the Laplacian the number of those products also grows with the mesh as the operator’s conditioning worsens (Saad, 2003). The reduced query time therefore grows more slowly with the mesh than the full-order time (Table 1).

When a rule is confirmed. A rule is an approximation, so we check it where it will be used, on states the reduced solver actually reaches and that were held out from the fit. On one state its error is the relative error ρ of the tested advection (17). A rule is *confirmed* when ρ stays below a bar fixed before certification on every held-out draw and on one further, independent confirmation draw (Table D.1). We never use the NNLS fit residual for this, because it measures only the fit states. Table 1 marks every row whose rule is not confirmed. The rule changes how the residual is evaluated, not what the model can represent, so it is a cost knob, separate from the accuracy knob R' (§6.2).

3.4 MODEL ARCHITECTURE

Three properties of the solve motivate the model in Figure 1. First, every head solve starts from a stored training code and descends along the head’s Jacobian, so that Jacobian must keep a usable direction; this motivates a *linear skip* in the head. Second, empirical quadrature keeps a sparse set of nodes and discards the rest, which saves time only if the decoder can be evaluated at one node without the others, and one trained model must serve every mesh of a series; this motivates a *coordinate-network bank*. Third, one trained model must offer several accuracy settings without retraining; this motivates an *ordered bank*. Widths and training settings are in Appendix C.

Linear skip, for the latent solve. The head is a shallow network plus a linear map,

$$h_\theta(z) = W^\top z + \varphi_\theta(z), \quad \varphi_\theta(z) = W_3 \sigma(W_2 \sigma(W_1 z + b_1) + b_2) + b_3, \quad (12)$$

with σ the SiLU activation. Its Jacobian is $Dh_\theta(z) = W^\top + D\varphi_\theta(z)$. The first term does not depend on z , so the Levenberg–Marquardt step from a stored code always has a fixed linear direction to follow, while φ_θ supplies the curvature of the manifold. The head acts on the R bank coefficients, not on the n nodes, so its cost and its parameter count do not depend on the mesh.

Coordinate-network bank, for node-local evaluation. Each column of the bank is a coordinate network read at the node and multiplied by the boundary factor of (1),

$$G_{x,r} = \mu(x) g_{\phi,r}(x), \quad g_\phi(x) = \text{MLP}_\phi(\gamma(x)), \quad \gamma(x) = [\cos(2\pi\Omega x), \sin(2\pi\Omega x)], \quad (13)$$

where γ is a random-Fourier-feature embedding with a fixed random frequency matrix Ω (Tancik et al., 2020). Because g_ϕ reads a coordinate, a row of G is computed from its node alone. A quadrature rule therefore needs only the rows of $\mathcal{S}^+$, and the same trained network is sampled on every mesh. Two natural alternatives fail one of these tests. A convolutional or attention decoder couples its output nodes, so one node costs a pass over the whole grid, and a learned dense readout stores nR parameters and must be retrained for each mesh. The parameter count of g_ϕ does not depend on n .

Ordered bank, for deployment-time accuracy. The rotation (2) puts the most useful directions first. Cutting the bank at R' then gives the best R' -dimensional subspace of the bank for the training fields in the least-squares sense, and the settings are nested, so one model built with R columns contains every smaller one. A query picks R' and no weight changes. The two solve modes of (3) are the two ends of the model. The span is a linear ROM in a learned bank, and on a linear PDE it is one least-squares solve. The head is a nonlinear-manifold ROM in k unknowns and can follow states that no small span contains. The alternatives, a separate model per accuracy or a retrained smaller bank, both need retraining. One frozen model supplies both settings of every row of Table 1.

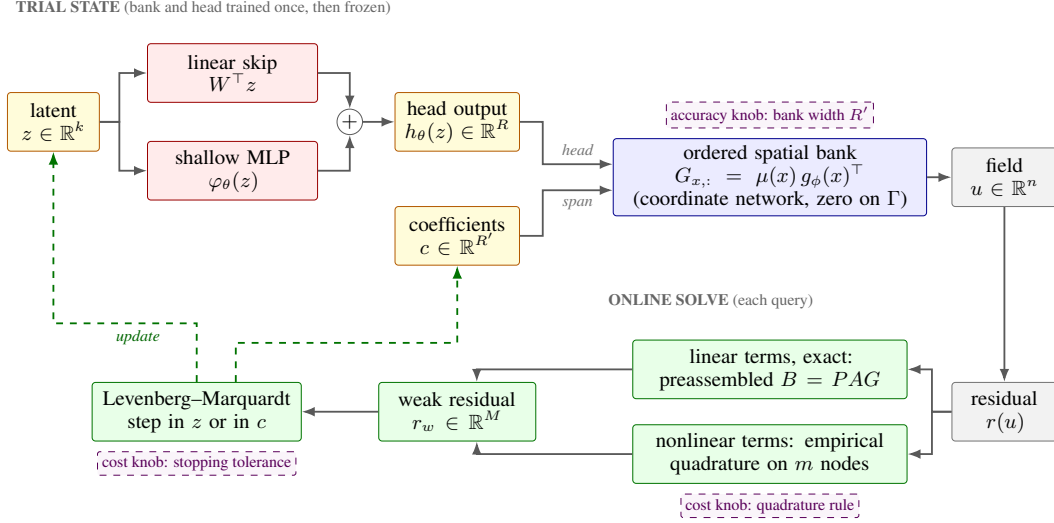


Figure 1: NM-ROM model and online solve. Top, trained once and then frozen: the latent code z passes through a linear skip and a shallow SiLU network, and their sum gives the bank coefficients $h_\theta(z)$. The ordered coordinate-network bank, multiplied by a factor that vanishes on the boundary Γ , turns coefficients into the field u with the Dirichlet data exact. A query uses either the head, $u = Gh_\theta(z)$, or the span of the first R' ordered columns, $u = \hat{G}_{R'}c$. **Bottom, solved at each query:** the residual of u is tested against M fixed weak tests. Linear terms use the preassembled matrix B and are exact; Burgers advection uses the empirical quadrature rule on m nodes. A Levenberg–Marquardt step updates z or c . Dashed tags mark the deployment knobs: the bank width R' sets the accuracy, and the quadrature rule and the stopping tolerance set the cost.

What is fixed and what is solved. Training fixes the bank, its ordering, the head and the tests for every row of Table 1. Each query solves z or c . Deployment chooses the mode and R' , the test count M , the quadrature and the stopping rule, and leaves the weights fixed.

4 IMPLEMENTATION

4.1 MATRIX-FREE PROJECTED OPERATORS VIA JAX

The matrix B of §3.3 is built once per mesh, so the online residual of a linear term is the product $Bc - b$. In the span the Jacobian is the fixed matrix $BT_{:,1:R'}$. With the head the remaining derivative is the Jacobian of the head, which forward-mode automatic differentiation (Bradbury et al., 2018) supplies,

$$Dh_\theta v = \text{JVP}(h_\theta, z, v), \quad J_z = B Dh_\theta(z). \quad (14)$$

Each accepted step solves the damped normal equation directly. A linear PDE does not return to the grid, and the projected operator is not passed to a Krylov method. Burgers advection reads the cached stencil on the quadrature nodes; runs above 1024^2 assemble its Jacobian analytically (Appendix A.3). The ordered bank is stored in column blocks at the tested widths, so a query at width R' reads exactly R' columns.

4.2 TRAINING PROTOCOL

Training is an auto-decoder on training data only, in two stages. The coordinate network g_ϕ is fitted to the training states through (1) and frozen, which fixes the bank. The head is then fitted to the bank coefficients of those states, with one stored code per training state. Last, the bank is ordered once by (2). This step has no trainable weights and takes one QR and one SVD. Widths, cohorts and offline costs are in Appendix C.

5 EXPERIMENTAL SETUP AND BENCHMARK PROBLEMS

Problems and references. In two dimensions we solve Poisson on the square, heat and viscous Burgers, with an L-shaped Poisson domain as an ill-conditioned check. In three dimensions we solve Poisson, heat and incompressible Navier–Stokes on the periodic cube, whose initial velocity is the curl of two localised vector potentials with random centre, separation, strength and viscosity. Meshes and cohorts are in Table C.2, the sampled families in Table C.3 and the three-dimensional configurations in Table C.9.

Accuracy. We report the worst relative L^2 error over the stated cases and requested times, in percent. Poisson uses the steady reference norm and heat the current reference norm; Burgers and Navier–Stokes use the initial-field norm over evolved times. Same-grid error measures the reduction error against the converged discrete solution. Error against a refined reference also includes discretisation error, and we never interchange the two. Settings for a held-out cohort are fixed before its cases are opened; all other cohorts are development cohorts.

Timing and speedup. Every accuracy–runtime pair comes from the same solver invocation, in double precision, with GPU burn-in, synchronised timing and medians of retained repetitions (protocol in Appendix C.1). GPU query time covers initialisation, the solve or rollout and the requested device outputs. Training and compilation are offline costs. Speedup is $S = T_{\text{FOM}}/T_{\text{method}}$ with both times from the same allocation, and each table names the FOM and shows its error. Solves that miss their stopping rule are reported as such and do not enter a speedup.

Baselines. Poisson, including the L-shaped check, is compared with conjugate gradients (CG), and heat with Crank–Nicolson stepping whose linear solve is CG (CN–CG). Burgers is compared with Newton–BiCGStab. Navier–Stokes is compared with the solver that generated its data, a Fourier pseudo-spectral method with Crank–Nicolson diffusion and Adams–Bashforth advection (CNAB2), which needs no linear solve. The named FOM of a row is that solver at its fastest tested setting that is at least as accurate as the accurate NM-ROM setting. Other nonlinear-manifold ROMs are compared in Table 2, and neural operators (FNO, U-Net, DeepONet and Transolver) in Appendix B.

6 NUMERICAL EXPERIMENTS AND RESULTS

Table 1 compares the NM-ROM with the named full-order solvers at every problem and mesh. The subsections that follow look at the bank-width knob behind the two settings of each row, and at which knob to turn.

6.1 ACCURACY AND SPEED AGAINST FULL-ORDER SOLVERS

Linear problems. On Poisson both settings of a row are solves in the span of the ordered bank. At 4096^2 the accurate width $R' = 512$ reaches 0.74 % at $106\times$ the speed of CG, and the fast width $R' = 128$ reaches 2.31 % at $358\times$. The head is less accurate here (3.15 %): on a linear problem the best solve in the bank is the linear one, and it is also the cheapest. The speedup grows with the mesh because the reduced query does not visit the grid until it returns the field. On the L-shaped check the accurate setting is $6.07\times$ faster at 2.12 %. A wide heat bank, evaluated once on a sealed held-out cohort at every mesh, reaches 0.49 % over all times and is $103\times$ faster than CN–CG at 4096^2 . In three dimensions the held-out Poisson cohort gives 0.26 % at $1.33\times$. The three-dimensional heat bank, evaluated once on a sealed cohort of 64 cases, meets the 1 % all-times target at every mesh (accurate 0.11 %) and overtakes CN–CG at 128^3 ($2.22\times$). **[L-shape, heat and 3D Poisson rows: bank-width reruns running; numbers above are the current checkpoint’s.]**

Burgers. **[Burgers rows still come from the earlier correction-direction solve; the bank-width rerun is running and replaces this paragraph.]** The reduced solve costs about the same at every mesh, so the fast setting overtakes Newton–BiCGStab by 1024^2 ($2.02\times$ faster at 2.29 % error). At 4096^2 the accurate setting chosen on six development cases reaches 0.60 % and is $4.87\times$ faster than the fastest Newton setting at least as accurate. On 64 held-out cases the same setting gives 1.33 % at $5.38\times$. The quadrature rule of those rows passes five held-out draws and fails the confirmation draw, so it is validated but not confirmed (Appendix C.1).

Table 1: NM-ROM against the named full-order solver at every problem and mesh. **Both NM-ROM columns of a row come from one frozen model:** *accurate* is the most accurate tested setting and *fast* the cheapest setting at least as accurate as the model’s earlier default (Table C.4); no network is retrained between them. Error is the worst relative L^2 error over the cases (%). Speedup is the FOM’s median time divided by the NM-ROM’s in the same GPU allocation, **bold** where the NM-ROM is faster. The FOM is the named solver at its fastest tested setting at least as accurate as the accurate setting, and both speedups of a row divide that one time. Navier–Stokes is compared with CNAB2, the Fourier pseudo-spectral solver that generated its data, which needs no linear solve; its fast setting is the cheapest bank width below 5 % error. Rows marked ^o use the ordered bank; the others still use the earlier correction settings until their reruns finish. Marker definitions are in Appendix C.1. Dashes mark settings not measured.

Problem	Mesh	NM-ROM accurate		NM-ROM fast		Full-order model (FOM)	
		Err. (%)	Speedup	Err. (%)	Speedup	Err. (%)	Solver
<i>Two-dimensional</i>							
Poisson ^o	256 ^{2p}	0.75	13.4 ×	2.31	17.5 ×	0.55	CG, rtol 3×10 ^{−2}
	1024 ^{2p}	0.74	27.9 ×	2.31	68.5 ×	0.22	CG, rtol 3×10 ^{−2}
	2048 ²	0.74	54.1 ×	2.31	174 ×	0.61	CG, rtol 10 ^{−1}
	4096 ²	0.74	106 ×	2.31	358 ×	0.45	CG, rtol 10 ^{−1}
Poisson, L-shape ^c	256 ²	2.13	3.84 ×	3.86	4.04 ×	0.64	CG, rtol 10 ^{−2}
	512 ²	2.12	6.07 ×	3.85	6.18 ×	0.38	CG, rtol 10 ^{−2}
	1024 ²	2.20	8.38 ×	3.85	8.59 ×	1.05	CG, rtol 3×10 ^{−2}
	2048 ²	2.20	17.0 ×	3.85	17.1 ×	0.76	CG, rtol 3×10 ^{−2}
Heat ^{tfa}	1024 ²	0.49	13.9 ×	1.36	13.0 ×	0.18	CN–CG, Δt=0.05, rtol 10 ^{−3}
	2048 ²	0.49	60.0 ×	1.36	57.0 ×	0.18	CN–CG, Δt=0.05, rtol 10 ^{−3}
	4096 ²	0.49	103 ×	1.36	101 ×	0.47	CN–CG, Δt=0.05, rtol 10 ^{−2}
Burgers ^e	256 ²	0.51 ^s	0.043×	1.89	0.79×	0.049	Newton–BiCGStab, tol 10 ^{−3}
	512 ²	0.55 ^x	0.068×	2.14	1.31 ×	0.052	Newton–BiCGStab, tol 10 ^{−3}
	1024 ²	0.59 ^d	0.0037×	2.29	2.02 ×	0.034	Newton–BiCGStab, tol 10 ^{−4}
	2048 ²	0.87 ^ℓ	0.99×	2.37	4.16 ×	0.049	Newton–BiCGStab, tol 3×10 ^{−3}
	4096 ²	0.60 ^w	4.87 ×	2.41	10.10 ×	0.050	Newton–BiCGStab, tol 3×10 ^{−3}
<i>Three-dimensional</i>							
Poisson ^f	32 ³	0.26	0.94×	1.40	0.99×	0.16	CG, rtol 10 ^{−2}
	64 ³	0.26	1.33 ×	1.39	1.38 ×	0.11	CG, rtol 10 ^{−2}
Poisson (dev. sources)	128 ³	0.16	6.75 ×	0.55	6.98 ×	0.075	CG, rtol 10 ^{−2}
	256 ³	0.16	23.2 ×	0.55	23.7 ×	0.049	CG, rtol 10 ^{−2}
Heat ^{tfa}	32 ³	0.11	0.62×	2.00	0.57×	0.078	CN–CG, Δt=0.025, rtol 10 ^{−4}
	64 ³	0.11	0.94×	2.00	0.85×	0.081	CN–CG, Δt=0.025, rtol 10 ^{−4}
	128 ³	0.11	2.22 ×	2.00	2.10 ×	0.082	CN–CG, Δt=0.025, rtol 10 ^{−4}
Navier–Stokes ^{eo}	32 ³	0.15	1.09 ×	2.96	1.87 ×	0.094	CNAB2 (spectral), 50 steps
	64 ³	0.15	2.89 ×	2.96	5.09 ×	0.093	CNAB2 (spectral), 50 steps
	96 ³	0.15	6.67 ×	2.96	11.6 ×	0.063	CNAB2 (spectral), 60 steps

Navier–Stokes. Here the head earns its place. At 96³ the head with $k=8$ reaches 0.15 % and is $6.67\times$ faster than CNAB2 at matched accuracy. The span of the same bank is $36\times$ less accurate at the same number of unknowns, and even the full rank-64 span reaches only 0.62 %. The moving vortices lie on a low-dimensional curved manifold that the head follows and a small linear span does not. The error stays flat from 32³ to 96³ while the speedup rises from $1.09\times$ to $6.67\times$. On 32 held-out cases the frozen settings give 0.21 % at $7.79\times$.

Resolution. In every series of Table 1 the NM-ROM query time grows more slowly with the mesh than the FOM’s, so the speedup rises with resolution while the error stays near its coarse-mesh value. The selected FOM setting may change with the mesh, since it must match the accurate setting at each one.

On 32 validation Burgers cases at 256² and 512² (Table 2) our fast setting, at matched latent dimension, reaches worst errors of 6.79–7.72 %, against 120–252 % for the reproduced shallow masked-autoencoder NM-ROM and 25–56 % for POD-LSPG. Our accurate setting reaches 0.88–1.05 %.
[add the quadratic-manifold row (QM16) and the bank-width rows from the high-resolution comparison.]

Table 2: Other nonlinear-manifold ROMs on the shared Burgers 2D family at 256^2 and 512^2 : worst and median evolved same-grid error over 32 validation cases (never used for any choice) and compiled-query memory, one allocation. Latent dimension is matched for the baselines and our fast setting. The shallow masked-autoencoder NM-LSPG of Kim et al. (2022) passed its reproduction gate on the third of three pre-registered attempts; its hyper-reduction was not reproduced, and its published encoder width does not fit our devices, so at 512^2 its width-capped encoder is limited by training time (Table C.7, which also states the data-matched row and the residual path). Every row here, ours included, uses a dense residual, so no times are given.

Method	Unknowns	256^2			512^2		
		Worst (%)	Median (%)	MB	Worst (%)	Median (%)	MB
Kim et al. NM-LSPG	8	163.93	43.34	2480	175.71	55.48	9959
Kim et al. NM-LSPG	16	145.40	43.87	2567	174.99	58.56	10511
Kim et al. NM-LSPG	32	127.51	43.61	2742	252.33	56.72	10879
Kim et al. NM-LSPG, data-matched	16	120.33	38.71	2567	—	—	—
POD-LSPG	8	55.52	22.72	18	55.82	24.08	71
POD-LSPG	16	41.53	10.45	30	42.01	13.83	122
POD-LSPG	32	24.78	5.65	55	25.63	7.74	222
This work, fast ($q = 0, k = 16$)	16	6.79	0.66	625	7.72	0.69	2368
This work, accurate ($q = 256, k = 16$)	272	0.88	0.089	1769	1.05	0.080	6891

Settings. Table 3 shows the whole ladder behind the two settings of a row. On Poisson at 4096^2 , narrowing the bank from $R' = 512$ to 32 raises the error $12.4\times$, from 0.74 to 9.17 %, and lowers the query time $9.65\times$, so the same model moves from $106\times$ to $1022\times$ the speed of CG. On Navier–Stokes the span ladder runs from 0.62 % at $R' = 64$ to 5.55 % at $R' = 8$, at $1.89\times$ lower cost, and the head sits above it in accuracy. Moving along a ladder never requires retraining, only a new deployment argument. This is what we mean by deployment-time tunability.

6.2 WHICH KNOB TO TURN

We sort the knobs of the framework into three groups (Table 4). The first is the *deployment-time accuracy* knob, the bank width R' together with the choice between head and span, which changes what the trained model can represent. The second holds the *deployment-time cost* knobs, the quadrature rule and the stopping tolerance, which change how the residual is evaluated and when the solve stops, at little change in error. The third holds *engineering* choices in the solver path, which do not change the answer but set how long it takes. The iteration cap is a safeguard, not a knob, since a truncated solve fails.

The first group is what makes the NM-ROM tunable in a way a trained neural operator is not: the same frozen model is reused, and only a deployment argument changes. Which end to use depends on the problem. On the linear problems the span wins at every width, so the knob is R' alone. On Navier–Stokes the head is the accurate end and the span ladder the fast end, and the head costs about twice the time of the span at the same number of unknowns, because each step also evaluates the network and its Jacobian.

The second group lowers the cost of a chosen accuracy. Empirical quadrature removes the mesh from the Burgers residual (§3.3), and a looser stopping tolerance saves time at the same error. Neither changes the trial space, so neither can buy accuracy.

The engineering group contributes most of the Burgers speedup at fine meshes. The solver path of the current rows (Cholesky factorisation of the damped normal equation, a clipped step, damping carried between steps, a quadratic predictor and an analytic Jacobian; Appendix A.3) reaches the same error as the path it replaced [**pending: 1024^2 gain**] $\times$ faster in one allocation. Each change is checked to reproduce the replaced path’s iterates and iteration counts.

Limitations. (i) Except for Navier–Stokes, speedups are against named iterative full-order solvers at the stated tolerances. Fast direct and spectral solvers exist for the square and the cube and would be faster than CG there. The named FOM is more accurate than the NM-ROM in every row. (ii) The Navier–Stokes bank is a POD basis in a moving frame, not a coordinate network, and the family has moderate Reynolds numbers; we make no claim about turbulent flow. (iii) On Poisson the errors lie near the bank floor (0.742 % square, 0.143 % cube), so a wider or better bank, not the solver, limits

Table 3: Bank-width tunability at the finest mesh. Each block is one frozen model, and within a cohort every speedup divides the median time of *one* full-order setting (the *FOM* row, the fastest at least as accurate as the block’s most accurate setting, in the same allocation) by the setting’s time (ms). Error is the worst same-grid relative L^2 error (%), over evolved times for Navier–Stokes, Burgers and heat. Held-out cases were never used for selection and share the development settings. **[Burgers and heat blocks still show the earlier correction ladder; their bank-width reruns are running.]**

Setting	Development cohort			Held-out cohort		
	Err. (%)	ms	Speedup	Err. (%)	ms	Speedup
<i>Poisson 2D, ordered bank, 4096² (12 development sources)</i>						
Head, $k=32$, $R'=512$	3.15	43.4	103×	—	—	—
Bank span, $R'=512$	0.74	42.4	106×	—	—	—
Bank span, $R'=384$	0.77	32.5	138×	—	—	—
Bank span, $R'=256$	0.94	22.6	199×	—	—	—
Bank span, $R'=128$	2.31	12.5	358×	—	—	—
Bank span, $R'=64$	5.33	7.1	631×	—	—	—
Bank span, $R'=32$	9.17	4.4	1022×	—	—	—
<i>FOM</i> : CG, $\text{rtol } 10^{-1}$	0.45	4490	1×	—	—	—
<i>Navier–Stokes 3D, co-moving frame, ordered bank, 96³ (16 development / 32 held-out cases)</i>						
Head, $k=8$	0.15	16.0	6.67×	0.21	15.8	7.79×
Bank span, $R'=64$	0.62	15.4	6.92×	1.12	15.3	8.05×
Bank span, $R'=48$	1.00	12.9	8.27×	1.26	12.6	9.76×
Bank span, $R'=32$	1.26	11.6	9.14×	1.46	11.5	10.7×
Bank span, $R'=16$	2.96	9.2	11.6×	3.24	9.2	13.4×
Bank span, $R'=8$	5.55	8.2	13.0×	6.91	8.1	15.2×
<i>FOM</i> : CNAB2 (spectral), 60 / 70 steps	0.063	106	1×	0.056	123	1×
<i>Burgers 2D, correction rank (6 development / 64 held-out cases)</i>						
$q = 0$, $M = 64$	2.41	40.7	12.9×	9.03	40.6	13.2×
$q = 128$, $M = 576$	1.09	77.4	6.77×	2.43	74.6	7.21×
$q = 256$, $M = 544$	0.88	112	4.67×	2.93	105	5.11×
$q = 256$, $M = 1088$	0.60	127	4.12×	1.33	113	4.77×
$q = 256$, $M = 1088$, looser tol.*	0.60	108	4.87×	1.33	100	5.38×
$q = 256$, $M = 2176^\dagger$	0.46	132	3.96×	1.13	126	4.25×
<i>FOM</i> : Newton–BiCGStab, tol. 3×10^{-3}	0.050	524	1×	0.14	537	1×
<i>Heat 2D, wide bank, correction rank (16 sealed held-out cases)</i>						
$q = 0$	—	—	—	0.84	32.2	146×
$q = 32$	—	—	—	0.22	31.7	149×
$q = 96$	—	—	—	0.082	24.4	193×
<i>FOM</i> : CN–CG, $\Delta t = 0.025$, $\text{rtol } 10^{-6}$	—	—	—	0.045	4713	1×

accuracy. The L-shape (2.20 % against a floor of 0.77 %) is limited by the head. (iv) Quadrature validation covers finitely many reached states and is not a global certificate. The 4096² Burgers rule is not confirmed on independent re-draws, and no rule is used in three dimensions. Reduced solves can converge to a wrong branch, and lower same-grid error does not remove discretisation error. (v) Unmarked cohorts are small development cohorts; exceptions are marked held-out. Timings report medians without dispersion.

7 CONCLUSION AND FUTURE WORK

We built an NM-ROM whose distinguishing property is a deployment-time accuracy/speed tradeoff from a single trained model: at inference, the width of the ordered bank, the choice between the head and the span, the quadrature rule and the stopping tolerance trade accuracy for speed. This is the lever neural operators do not expose, since each trained FNO or DeepONet delivers one fixed operating point. Mechanically, the decoder is a learned manifold built from a coordinate-network bank and a nonlinear head; the PDE itself is enforced by least-squares Petrov–Galerkin projection of the discrete residual onto that manifold; Dirichlet boundary conditions are enforced exactly by the bank; linear

Table 4: Which knob to turn, measured from frozen models. Bank width: Table 3. EQ: dense time over EQ time for Burgers in one allocation per mesh. Tolerance and cap: 32 validation Burgers cases (Table D.2), stopping tolerance $10^{-8} \rightarrow 10^{-3}$ and iteration cap 2. The engineering row leaves the iterates unchanged. * Jobs 3789570 and 3789572.

Knob	Turn it to	Effect on error	Effect on cost
<i>Deployment time, accuracy (no retraining):</i>			
Bank width R' , Poisson 4096 ²	512 $\rightarrow$ 32	12.4 $\times$ higher	9.65 $\times$ lower
Bank width R' , Navier–Stokes 96 ³	64 $\rightarrow$ 8	0.62 $\rightarrow$ 5.55 %	1.89 $\times$ lower
Head instead of span, Navier–Stokes 96 ³	$R' = k = 8$	36 $\times$ lower	higher (Table 3)
<i>Deployment time, cost (no retraining):</i>			
Empirical quadrature	dense $\rightarrow$ stored rule	Table C.8	4.77 $\times$ lower at 256 ² , 38.53 $\times$ at 1024 ² *
Stopping tolerance	looser	6.712 $\rightarrow$ 6.701 %	35.6 % lower
<i>Engineering (same answer):</i>			
Solver path	before $\rightarrow$ after	unchanged	[pending: 1024 ² gain, job 4187625] lower
Safeguard:			
Iteration cap	2 iterations	fails (90.3 %)	—

terms are preassembled exactly and empirical quadrature removes the mesh from the nonlinear term; and the whole pipeline is matrix-free in JAX.

Three pieces had to come together to make this work. First, a coordinate-network bank, which is evaluated at one node without the others and serves every mesh of a series from one trained network. Second, a one-time ordering of that bank, which turns one trained model into a nested family of trial spaces chosen at the query. Third, a head with a linear skip, which gives the latent solve a usable direction from a stored code and supplies the accurate end where a linear span is not enough, as on Navier–Stokes.

The model keeps its accuracy as the mesh is refined, so its speedup over the named solvers grows with resolution. At 4096² it reaches 0.74 % on Poisson at 106 $\times$ the speed of CG, and on three-dimensional Navier–Stokes at 96³ the head reaches 0.15 % at 6.67 $\times$ the speed of a spectral solver, 36 $\times$ more accurate than a linear span of the same size. The named solvers remain more accurate. A natural next step is a coordinate-network bank for moving-frame flows, quadrature in three dimensions, and advection-dominated three-dimensional problems.

REPRODUCIBILITY STATEMENT

Every table and prose number is generated from hash-pinned run records identifying source, checkpoint, settings, cohort, allocation and audit (Appendix C); failed settings are kept.

AI USE STATEMENT

Language-model assistants supported method and experiment development, implementation, analysis, auditing and drafting. Recorded solver runs produced the numbers and the generators reproduce them. The authors are responsible for the methods, results and text.

REFERENCES

- J. Barnett and C. Farhat. Quadratic approximation manifold for mitigating the Kolmogorov barrier in nonlinear projection-based model order reduction. *Journal of Computational Physics*, 464:111348, 2022.
- P. Benner, S. Gugercin, and K. Willcox. A survey of projection-based model reduction methods for parametric dynamical systems. *SIAM Review*, 57(4):483–531, 2015.
- J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han. Once-for-All: Train one network and specialize it for efficient deployment. In *International Conference on Learning Representations*, 2020.

- K. Carlberg. Adaptive h -refinement for reduced-order models. *International Journal for Numerical Methods in Engineering*, 102(5):1192–1210, 2015.
- K. Carlberg, C. Bou-Mosleh, and C. Farhat. Efficient non-linear model reduction via a least-squares Petrov–Galerkin projection and compressive tensor approximations. *International Journal for Numerical Methods in Engineering*, 86(2):155–181, 2011.
- A. Cohen and R. DeVore. Approximation of high-dimensional parametric PDEs. *Acta Numerica*, 24:1–159, 2015.
- R. Geelen, S. Wright, and K. Willcox. Operator inference for non-intrusive model reduction with quadratic manifolds. *Computer Methods in Applied Mechanics and Engineering*, 403:115717, 2022.
- G. H. Golub and V. Pereyra. The differentiation of pseudo-inverses and nonlinear least squares problems whose variables separate. *SIAM Journal on Numerical Analysis*, 10(2):413–432, 1973.
- C. Greif and K. Urban. Decay of the Kolmogorov N -width for wave problems. *Applied Mathematics Letters*, 96:216–222, 2019.
- J. A. Hernández, M. A. Caicedo, and A. Ferrer. Dimensional hyper-reduction of nonlinear finite element models via empirical cubature. *Computer Methods in Applied Mechanics and Engineering*, 313:687–722, 2017.
- M. R. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436, 1952.
- Y. Kim, Y. Choi, D. Widemann, and T. Zohdi. A fast and accurate physics-informed neural network reduced order model with shallow masked autoencoder. *Journal of Computational Physics*, 451:110841, 2022.
- M. Kim, T. Wen, K. Lee, and Y. Choi. Physics-informed reduced order model with conditional neural fields. *arXiv preprint arXiv:2412.05233*, 2024.
- N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar. Neural operator: learning maps between function spaces. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- C. L. Lawson and R. J. Hanson. *Solving Least Squares Problems*. Prentice-Hall, Englewood Cliffs, NJ, 1974.
- K. Lee and K. T. Carlberg. Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. *Journal of Computational Physics*, 404:108973, 2020.
- Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.
- L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.
- D. W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963.
- B. Peherstorfer and K. Willcox. Online adaptive model reduction for nonlinear systems via low-rank updates. *SIAM Journal on Scientific Computing*, 37(4):A2123–A2150, 2015.
- O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.
- Y. Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, 2nd edition, 2003.

- L. Sirovich. Turbulence and the dynamics of coherent structures, Parts I–III. *Quarterly of Applied Mathematics*, 45(3):561–590, 1987.
- R. Ștefănescu and A. Sandu. Efficient approximation of sparse Jacobians for time-implicit reduced order models. *arXiv preprint arXiv:1402.2018*, 2014.
- M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, and M. Niepert. PDEBench: An extensive benchmark for scientific machine learning. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 35, 2022.
- M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in Neural Information Processing Systems*, 33, 2020.
- J. Weder, P. Schwerdtner, and B. Peherstorfer. Online-efficient model reduction on quadratic manifolds with Neural Galerkin schemes. *arXiv preprint arXiv:2412.17695*, 2024.
- H. Wu, H. Luo, H. Wang, J. Wang, and M. Long. Transolver: A fast transformer solver for PDEs on general geometries. In *International Conference on Machine Learning*, 2024.
- M. Yano and A. T. Patera. An LP empirical quadrature procedure for reduced basis treatment of parametrized nonlinear PDEs. *Computer Methods in Applied Mechanics and Engineering*, 344:1104–1123, 2019.
- J. Yu, L. Yang, N. Xu, J. Yang, and T. Huang. Slimmable neural networks. In *International Conference on Learning Representations*, 2019.

A METHOD DETAILS

This appendix writes the reduced problem (4) out for each PDE: what the residual is, how the time step enters, and what is solved at each query. Formulas are for the scalar two-dimensional Dirichlet case. The three-dimensional problems are in Appendix C.1 and Navier–Stokes in §A.4.

A.1 ELLIPTIC INSTANCE: POISSON

For $Au = f$ we take $\Lambda_\star = \Lambda$. Testing the residual and rescaling the rows leaves (5). Because the retained tests are orthonormal and $Au^\star = f$, this residual is exactly the tested error $P(u - u^\star)$, so the minimised objective is the squared discrete L^2 error of the reduced state in the retained modes; this holds only for the modes P retains.

Span solve. In the span of the first R' ordered columns the residual (5) is linear, with the fixed matrix $B_{R'} = B_0 T_{:,1:R'}$. Let $B_{R'} = QR$ be a thin QR factorisation, computed once per mesh and width. The query is then $\mathcal{R}c = Q^\top b_0$, one matrix–vector product and one triangular solve, with no iteration. At $R' = R$ this is the solve in the full bank, whose error is the bank floor in the tested norm. The head solve iterates Levenberg–Marquardt on z with the exact Jacobian $B_0 Dh_\theta$. We report the stationarity of the head solve and the backward error and numerical rank of the triangular solve.

A.2 PARABOLIC INSTANCE: HEAT

The reduced heat step is one nonlinear least-squares problem per time step. The semi-discrete equation $\dot{u} = -\kappa Au$ is advanced with Crank–Nicolson, $(I + \frac{\Delta t \kappa}{2} A)u^{n+1} = (I - \frac{\Delta t \kappa}{2} A)u^n$, and the reduced step substitutes the manifold into the fully discrete equation *before* projecting. With $u^n = u(z_n)$, projecting with P and row-scaling by $\Lambda_\star = I + \frac{\Delta t \kappa}{2} \Lambda$ gives (6), where $D = \text{diag}((1 - \frac{\Delta t \kappa}{2} \lambda_i)/(1 + \frac{\Delta t \kappa}{2} \lambda_i))$, warm-started at z_n . Two points are structural. The right-hand side is built from the *decoded current state* at every step: carrying the quantity the previous step already made small freezes the recursion and reproduces a one-step solution to round-off. And z_{n+1} enters through h_θ , so the step is a nonlinear least-squares problem, not a linear solve. In the span the step is linear in the coefficients, and it is the linear least-squares recurrence

$$c_{n+1} = B_0^+ D B_0 c_n, \quad (15)$$

with B_0 replaced by $B_{R'}$, $B_{R'}^+$ the pseudo-inverse ($M \geq R'$; the numerical rank is checked) and c_0 fitted to the supplied initial field. This is the weak Crank–Nicolson propagation of the bank coefficients, with no head and no iteration. Because the heat equation is linear and autonomous and the tests are eigenfunctions, the coefficients at any output time can also be fitted directly to exactly propagated test moments; we call this the batched fit. The deployed path factors the damped normal matrix by Cholesky; a variant assembles the Gram matrix $S = B_0^\top B_0$ offline and forms $H = Dh_\theta^\top S Dh_\theta$ directly, which is exact and is reported as an algebraic ablation of the same step.

A.3 NONLINEAR ADVECTION: BURGERS

Burgers is the only problem whose tested residual stays nonlinear in the coefficients, so nothing is eliminated in closed form. The full-order problem is $u_t + u(u_x + u_y) = \nu \Delta u$ with the non-conservative sign-upwind stencil

$$N(u)_{ij} = u_{ij} (\delta_x u + \delta_y u)_{ij}, \quad (\delta_x u)_{ij} = N \cdot \begin{cases} u_{ij} - u_{i-1,j}, & u_{ij} > 0, \\ u_{i+1,j} - u_{ij}, & u_{ij} \leq 0, \end{cases} \quad (16)$$

δ_y analogous and switching on the same centre value, ghost zeros on all four walls, and backward Euler in time, $r_n(u) = u - u^n + \Delta t (N(u) - \nu \Delta_h u)$. Projecting and row scaling by $\Lambda_\star = I + \Delta t \nu \Lambda$ gives (7), with $c = h_\theta(z)$ for the head and c the span coefficients otherwise. The diffusion term is exact and the row scaling is the diagonal of the linearised implicit operator in the modal basis, the modal form of the Helmholtz preconditioner the full-order Newton solver uses. Only $\hat{N}(c) \approx PN(Gc)$ requires a choice: dense evaluation at every interior node, the sampled rule $\hat{N}(c) = \sum_{i \in \mathcal{S}} w_i P_{:,i} N_i(Gc)$ with the sign switch retained at the retained nodes, or the precomputed quadratic form $\frac{1}{2} c^\top Q c$ with Q

the symmetrised tensor $T_{iab} = \sum_x P_{ix} G_{xa} (DG)_{xb}$, which is an identity only where $u > 0$ at every stencil node of every trial state the solver visits. The last is an algebraic route we verified in-job (residual and Jacobian parity below 10^{-11} relative on sign-satisfying states) and do not time in this paper. Time stepping uses a fixed substep count per output interval; each step is warm-started at the previous code, and the linear extrapolation is used instead when, and only when, its residual norm is smaller; the 4096² rows also try a quadratic predictor under the same rule. The runs above 1024² assemble the Jacobian analytically instead of by forward-mode products and clip the step; in the settings listed in Table C.4 they solve the normal system by Cholesky factorisation. In every Burgers arm above 1024² except the 2048² development accurate arm, the damping λ is carried from one step to the next rather than reset to λ_0 .

A.4 INCOMPRESSIBLE FLOW: NAVIER-STOKES

The unknown is the three-component velocity on the periodic unit cube, with zero mean, $\partial_t u + (u \cdot \nabla)u = -\nabla p + \nu \Delta u$, $\nabla \cdot u = 0$. The initial velocity is the curl of two localised periodic vector potentials, each of the form $\exp[2 \sum_i (\cos(2\pi(x_i - c_i)) - 1)]$ along a fixed direction, with a random common centre, separation in $[0.12, 0.24]$, relative strength in $[0.6, 1.4]$ and viscosity log-uniform in $[0.002, 0.01]$. The horizon is $T = 0.2$ with six output times. The data and the full-order comparator come from a Fourier pseudo-spectral Galerkin method with two-thirds dealiasing, Crank–Nicolson diffusion and second-order Adams–Bashforth advection (CNAB2). Diffusion and the Leray projection are diagonal in Fourier space, so CNAB2 performs no linear solve.

The flow carries its vortices across the domain, so a fixed bank sees a translated copy of every structure. Writing $u(x, t) = v(x - s(t), t)$ and using that the quadratic term, the Laplacian and the Leray projector commute with translation gives $\partial_t v = \dot{s} \cdot \nabla v + \mathcal{P}[N(v)] + \nu \Delta v$. The bank G is a rank-64 POD of training states, each centred on its own energy centroid, and $v = Ga$. With fixed solenoidal Fourier tests the per-step residual has the terms of the fixed-frame model plus one term that is linear in the frame increment $\delta = s^{n+1} - s^n$. All reduced tensors, including the quadratic term, are computed offline, so no quadrature is needed. The head of (12) is trained as an auto-decoder on the centred bank coefficients of training states only, and the span uses the ordered bank of (2). A query centres the initial field, projects it on the bank and starts from the nearest stored code. It then takes a fixed number of damped Gauss–Newton sweeps per step in (z, δ) or (c, δ) , with an analytic Jacobian and a Cholesky solve. With the frame held fixed, the same bank misses every case by far more than the 5 % target, so the moving frame is necessary.

A.5 EMPIRICAL QUADRATURE: THE FIT AND THE COUNTS

A rule is judged by how well its weighted sum over m nodes reproduces the tested advection term of the full grid. That relative error, on one state, is

$$\rho(u) = \frac{\|\sum_{i \in \mathcal{S}} w_i P_{:,i} N_i(u) - PN(u)\|_2}{\|PN(u)\|_2}, \quad \rho_{\max} \leq 0.116 \text{ (primary bar)}, \quad \rho_{\max} \leq 0.06 \text{ (tight)}, \quad (17)$$

over the held-out reachable states described below. The primary bar is the held-out ρ of the original model’s head-only rule at the state that carries its largest first-interval error, measured in an earlier diagnostic study and fixed before any certification job ran; the tight bar was fixed before the EQ-ladder job of Table D.1. A candidate node set $\mathcal{C}$ is drawn once with a fixed seed. For each of n_{fit} stored codes $c^{(\ell)}$ we evaluate the decoded state, form the per-node contributions $N_i(Gc^{(\ell)})$ and the exact full-grid target $PN(Gc^{(\ell)})$, and solve

$$\min_{w \geq 0} \|\mathcal{G} w - \beta\|_2, \quad \mathcal{G}_{(\ell,i),c} = P_{i,c} N_c(Gc^{(\ell)}), \quad \beta_{(\ell,i)} = [PN(Gc^{(\ell)})]_i, \quad (18)$$

rows normalised by their Euclidean norms, support grown greedily by the Lawson–Hanson criterion with an exact non-negative refit after each addition and a hard cap of m nodes, padded to the requested count if the support saturates. Three counts are separated: the requested node count m , the achieved support, and the number of fit states n_{fit} . In the certification study the fit states are reachable states (states of the dense solver’s own converged per-step trajectory on fit trajectories), the held-out states are 512 reachable states per rung from certification trajectories disjoint from both the fit and the evaluation cases, and ρ of (17) is evaluated on those. A deployed rule is a stored triple: node indices, weights, and the cached $m \times 5 \times R$ bank block.

A.6 SOLVER CONSTANTS AND EXIT CODES

The representation ablations separate three error quantities: the *bank floor*, the projection error onto range G , which no head or solver can beat; the *best-found* error, the smallest any point of the manifold attains, located by a multistart oracle; and the *solved* error, what the deployed iteration actually returns.

Write J for the Jacobian of r_w in the unknowns ξ . Each Levenberg–Marquardt attempt solves $(H + \lambda \text{diag}(\text{diag } H)) \delta = -g$ with $H = J^\top J$ and $g = J^\top r_w$, and accepts δ only if the residual strictly decreases. The stationarity measure of §3.2 is scale free: it divides the gradient norm by the sizes of the Jacobian and the residual,

$$\eta(\xi) = \frac{\|J^\top r_w\|_2}{\|J\|_F \|r_w\|_2} \leq \eta_{\text{tol}}, \quad (19)$$

so one tolerance η_{tol} serves every mesh and every rank. The elliptic solve starts from the cached training code nearest the projected source; the heat and Burgers queries start from the nearest codes and fit z (or, in the span, c) to the supplied initial field.

Damping starts at $\lambda_0 = 10^{-6}$ (Poisson, Burgers) or 10^{-4} (heat); $\lambda \leftarrow \max(\lambda/3, 10^{-12})$ on acceptance and $\min(10\lambda, \lambda_{\text{max}})$ on rejection; the trust radius is taken from the spread of the training codes. Exit codes differ per problem and are reported per problem. Burgers: 4 stationarity, 1 residual, 2 tiny step, 3 rejected at $\lambda \geq 10^{14}$, 0 budget. Poisson: 6 stationarity, 2 relative residual, 1 stall, 3 damping limit, 5 non-finite initial value, 0 budget. Heat: 1 stationarity, 2 tiny step, 3 damping limit, 4 non-finite, 0 budget. The heat criterion divides by $\|J\|_F$ only, applied to a residual already normalised by the target norm; Poisson and Burgers use (19) directly. An initial fit that meets its stopping rule is accepted as the starting state of the time loop; this acceptance rule was fixed before the evaluation jobs ran, and complete exit flags remain in the source records.

B NEURAL OPERATORS

We trained an FNO (Li et al., 2021), a U-Net (Ronneberger et al., 2015), a DeepONet (Lu et al., 2021) and a Transolver (Wu et al., 2024) on the Burgers 2D family, on the same data split, metric, optimiser, seed and wall budget. They produce the requested output times in one pass, so their output times are fixed at training. An operator’s own evaluation resolution can also trade accuracy for cost (§2); here each trained operator has one evaluation setting. [\[higher-resolution operator comparison \(Burgers and heat\) from the running lanes goes here.\]](#)

Burgers at 256². The Burgers operators were also timed against the NM-ROM, POD and the full-order solver in one allocation (job 4179247, 6 development cases, 5 retained repetitions each; Table B.2). The fast operators are quicker than the NM-ROM and than the full-order setting the rule matches to them (3 of 9 operator arms beat it, up to $1.48\times$), and an order of magnitude less accurate: 4.46–7.42 % worst evolved error, against 0.51 % for the accurate NM-ROM setting and 1.89 % for the fast one. At this mesh no NM-ROM or POD arm is faster than its comparator; the speedups of the NM-ROM appear at the finer meshes of Table 1. Two asymmetries favour the operators and are left in place: their query is not charged the upload of the input field, and the fields their errors are scored on come from an extra, untimed query.

How much data each side was given. The operators were trained on 128 trajectories and our model on 4 608. The gap comes from the cost of one sample, not from a choice. Both sides draw initial conditions from the same family and are scored at the same output times, but each operator target is a 4096-interval solution restricted to the 256-interval grid, which costs about 670 times more than a trajectory behind our bank and head. Matching 4 608 trajectories at that fidelity would cost about 163 A100-hours. An operator also sees one input–output pair per trajectory (640 supervised states), while the head is fitted per state (131 072 states).

DeepONet. DeepONet was trained on the same terms as the other three families (job 4179556; Table B.1). Its 4 arms reach 14.79–18.22 % mean validation error, against 1.35–2.28 % for the selected arms of the other families, and 16.02–26.63 % worst error on the matched eight cases, where the NM-ROM reaches 1.87 %. A persistence control, which returns the input field at every output time, scores 64.68 %. All arms ended by early stopping with the training loss still falling, and no DeepONet-specific search was run, so we do not read these errors as an architecture ceiling. This lane took no timing.

Table B.1: DeepONet on Burgers 2D at 256² (job 4179556), beside the selected arm of each other family, the NM-ROM on the matched cohort and a persistence control. Fixed-initial relative error over the six requested output times, on the 32 validation cases and on the eight matched cases. Accuracy only: this lane takes no timing, and accuracy is comparable across these jobs while timing is not. “Ended by” is the recorded stop reason of each arm.

Arm	Ended by	Validation, 32 cases			Matched 8
		Mean (%)	Median (%)	Worst (%)	Worst (%)
DeepONet, small (selected)	early stopping	14.79	11.75	54.74	16.02
DeepONet, medium	early stopping	14.90	12.73	51.79	18.98
DeepONet, refined	early stopping	15.76	12.46	58.98	21.37
DeepONet, large	early stopping	18.22	14.80	60.35	26.63
U-Net, refined	wall budget	1.35	1.08	7.52	1.71
Transolver, refined	wall budget	1.95	1.36	9.32	1.52
FNO, large	wall budget	2.28	1.81	6.38	2.48
NM-ROM (same cohort)	—	—	—	—	1.87
Persistence control	—	64.68	65.52	90.65	75.00

Table B.2: Burgers 2D at 256^2 in one allocation (job 4179247): worst evolved and all-times same-grid error, median GPU query time, the full-order setting the rule matches to each row, and the speedup over it. Operator arms are the trained sizes of each family; the last three rows are the full-order settings the rule selects. Complete-query times add host transfer and are in the lane record.

Arm	Family	Evolved (%)	All times (%)	GPU ms	FOM by the rule	Speedup
NM-ROM, fast	NM-ROM	1.89	2.56	38.6	nt1e-3_dt01	0.50×
NM-ROM, accurate	NM-ROM	0.51	0.91	465	nt1e-3_dt005	0.07×
POD-LSPG, 256 modes	POD-LSPG	0.71	3.77	898	nt1e-3_dt005	0.03×
POD-LSPG, 512 modes	POD-LSPG	0.22	0.61	2770	nt1e-3_dt005	0.01×
FNO	FNO	7.42	7.42	7.3	nt1e-2_dt01	1.18×
U-Net, small	U-Net	5.14	5.14	9.4	nt1e-2_dt01	0.92×
U-Net, medium	U-Net	4.76	4.76	5.8	nt1e-2_dt01	1.48×
U-Net, large	U-Net	4.49	4.49	10.0	nt1e-2_dt01	0.86×
U-Net, refined	U-Net	4.55	4.55	5.9	nt1e-2_dt01	1.48×
Transolver, small	Transolver	4.90	4.90	11.0	nt1e-2_dt01	0.78×
Transolver, medium	Transolver	5.96	5.96	12.8	nt1e-2_dt01	0.67×
Transolver, large	Transolver	5.83	5.83	15.3	nt1e-2_dt01	0.57×
Transolver, refined	Transolver	4.46	4.46	11.1	nt1e-2_dt01	0.78×
nt1e-2_dt01	FOM	3.20	3.20	8.7	—	1×
nt1e-3_dt01	FOM	1.52	1.52	19.4	—	1×
nt1e-3_dt005	FOM	0.049	0.049	30.7	—	1×

Table B.3: Recorded training budgets of the neural operators: parameters, step budget, steps completed, the step selected by validation, batch, learning rate, seed and exit reason.

PDE	Operator	Parameters	Budget	Done	Selected	Batch	LR	Seed	Exit
Burgers 3D	fno	1771431	20000	20000	16100	2	0.001	920310	steps
Burgers 3D	unet	89431	20000	20000	17500	2	0.001	920311	steps
Burgers 3D	deepnet	1861959	30000	30000	30000	2	0.001	920312	steps
Burgers 3D	transolver	564392	30000	30000	29400	2	0.001	920313	steps
Heat 3D	fno_w16_m6	1771349	20000	20000	19500	4	0.001	920321	steps
Heat 3D	unet_w8	89197	20000	20000	20000	4	0.001	920322	steps
Heat 3D	deepnet_r128_w16	857749	20000	20000	20000	4	0.001	920323	steps
Heat 3D	transolver_w48_s32	562840	20000	20000	20000	4	0.001	920324	steps
Poisson 3D	fno_w24_m8	9440953	30000	30000	26750	2	0.0005	920450	steps
Poisson 3D	unet_w16	354577	30000	30000	28750	2	0.0005	920451	steps
Poisson 3D	deepnet_r128_w16	791697	30000	30000	29000	2	0.001	920452	steps
Poisson 3D	transolver_w48_s32	561272	30000	30000	29750	2	0.001	920453	steps
Navier-Stokes 3D	deepnet	6790319	32000	32000	23200	2	0.001	202609208	steps
Navier-Stokes 3D	transolver	569064	32000	32000	32000	2	0.001	202609209	steps
Navier-Stokes 3D	fno	4196511	8000	8000	7100	2	0.001	202609205	steps
Navier-Stokes 3D	unet	89287	8000	8000	7900	2	0.001	202609206	steps

C EXPERIMENTAL CONFIGURATION AND REPRODUCIBILITY

Tables C.2 and C.3 identify the two-dimensional studies and Table C.9 the three-dimensional ones. Within each study, training precedes evaluation, and the selected bank, its ordering and the head remain frozen. Deployment changes reduced coordinates, not network weights. Recorded training configurations and available offline costs are retained with the source evidence; unrecorded training costs are not inferred. The wide heat bank of Table 1 is a random-Fourier-feature coordinate network (32 features, scale 1.5, width 256, $R = 128$) trained for 80000 steps (learning rate 0.001, 64 states $\times$ 2048 points per batch, gradient clip 1, exact coefficient refit every 10000 steps), then a head of width 256 with $k = 8$ for 80000 steps (learning rate 0.0005, 128 states per batch), on 512 training draws at 64^2 with 16 validation draws; its validation projection floor is 0.11 %.

Table C.1: Training configurations of the frozen checkpoints, transcribed from the committed configuration and code files of each study (paths and SHA256 in evidence/training-configs-2026-09-21).

Model	Stage	Network	Optimiser, learning rate	Steps	Batch	Training data
Burgers 2D ($k = 16, R = 512$)	bank	128 Fourier features (scale 4), width 1024, 2 layers	AdamW, wd 10^{-5} , warm-up + cosine $10^{-3} \rightarrow 10^{-5}$	300000	all states $\times$ 4096 points	16384 states of 576 trajectories
	head	width 512, 2 layers	Adam, warm-up + cosine $10^{-3} \rightarrow 10^{-5}$	200000	4096 states	131072 states of 4608 trajectories
Poisson 2D ($k = 32, R = 512$)	bank	64 Fourier features (scale 4), width R , 2 layers	Adam, four phases at 10^{-3} , 3×10^{-4} , 3×10^{-5} , 10^{-4}	100000, 40000, 30000, 30000 (full batch)	64 sources $\times$ 4096 points	2611 fit / 461 validation sources
	head	width 128, 2 layers	Adam, 10^{-3}	150000	all	as bank
Poisson 2D ($k = 16, R = 128$)	bank + head	64 Fourier features, g and h width 128, 2 layers	Adam; staged from an $R = 64$ model (bank, head, joint at 3×10^{-4} , 3×10^{-5} , 10^{-4}) as Poisson $k = 32$	40000, 30000, 20000 after widening	64 sources $\times$ 4096 points	512 training sources
L-shape ($k = 16, R = 512$)	bank / head	64 Fourier features (scale 4), 2 layers; head width 128, 2 layers	Adam, cosine (floor 0.03); 10^{-3} bank, 5×10^{-4} head	100000, 40000, 30000, 30000; head 150000	64 sources $\times$ 4096 points	2611 fit / 461 validation sources
Poisson 3D ($k = 16, R = 128$)	bank / head	64 Fourier features (scale 1.5), widths 256; head width 256 + skip	Adam, cosine (floor 0.03); 10^{-3} bank, 5×10^{-4} head; clip 1, refit every 10000	150000 / 100000	64 states $\times$ 2048 points / 128 states	512 training / 16 validation fields
Heat 3D, earlier bank ($k = 32, R = 128$)	bank / head	32 Fourier features (scale 1.5); head width 256 + skip	Adam, cosine (floor 0.03); 10^{-3} bank, 5×10^{-4} head; clip 1, refit every 10000	150000 / 150000	64 states $\times$ 2048 points / 128 states	512 training / 16 validation trajectories

Full-order and solver details. The Burgers FOM is Newton’s method with BiCGStab inner solves (at most 200 iterations, Jacobian–vector products by forward-mode differentiation) preconditioned by the exact inverse of the implicit Helmholtz operator $(I + \Delta t \nu \lambda)^{-1}$ (advection dropped); Newton stops when the residual norm falls below the nonlinear tolerance times the previous state’s norm, or after 20 iterations, and always takes the full step (no line search or damping). Each named setting fixes Δt , the nonlinear and the linear tolerance. The L-shaped domain is an ill-conditioned Poisson check, not a separate method. The quadratic predictor of the 4096^2 Burgers rows forms the linear extrapolation $2w_n - w_{n-1}$ and, from the third step on, the quadratic one $3w_n - 3w_{n-1} + w_{n-2}$, evaluates the residual norm of the current code and both candidates in one batched call and starts the solve from the smallest.

Table C.2: Problem specification. Cohort and reduced sizes are read from the run configurations where recorded; rows describe their recorded development, validation and held-out cohorts.

PDE	equation, domain, boundary	meshes	time stepping	reduced sizes	reference	cohorts
Burgers 2D	$u_t + u(u_x + u_y) = \nu \Delta u$, $(0, 1)^2$, $u _{\partial\Omega} = 0$	256^2 – 4096^2	$\Delta t = 0.005$, backward Euler, sign-upwind	$k = 16, R = 512$	refined 4096^2 , $\Delta t = 0.00015625$	6 development cases; 32 validation; 64 held-out at 2048^2 , 4096^2 ; sealed cohort opened once
Poisson 2D	$-\Delta u = f$, $(0, 1)^2$, $u _{\partial\Omega} = 0$	256^2 – 4096^2	none (elliptic)	$k = 16, R = 128$ (original); $k = 32, R = 512$	exact discrete solution; 2048^2 refinement	12 development sources
Heat 2D	$u_t = \kappa \Delta u$, $(0, 1)^2$	64^2 – 1024^2	Crank–Nicolson	$k = 8, R = 32$	refined-grid reference ($1024^2/2048^2$ pair); error includes discretisation	12 development cases, 3 repetitions; measured in job 3529772; checkpoint lineage: earlier cell, job 3511417
Heat 2D, wide bank	$u_t = \kappa \Delta u$, $(0, 1)^2$	1024^2 – 4096^2	Crank–Nicolson; batched exact-propagator fit	$k = 8, R = 128$	exact same-grid propagation	12 development cases; 16 sealed held-out cases, opened once
Wave 2D (reflective)	$u_{tt} = c^2 \Delta u$, $(0, 1)^2$, $u _{\partial\Omega} = 0$	64^2 , 256^2 , 1024^2	RK4 on the manifold; exact modal propagation for the bank	$k = 32, R = 64$	exact modal solution	8 development cases
Poisson, L-shape	$-\Delta u = f$, $(0, 1)^2 \setminus [\frac{1}{2}, 1)^2$	256^2 – 2048^2	none	$k \in \{16, 32\}$, $R \in \{256, 512, 514\}$	converged discrete solution	3072 / 256 / 32 sources (train / selection / development)

C.1 HEADLINE SETTINGS, TIMES AND TIMING PROTOCOL

Table 1 markers. o ordered-bank settings of one frozen model, no retraining: Poisson accurate/fast = bank span $R' = 512$ / $R' = 128$ of the importance-ordered bank; Navier–Stokes accurate =

Table C.3: Sampled problem families, transcribed from the generator sources named in the last column (code constants, not run outputs).

PDE	equation and boundary	sampled family (transcribed from the generator source)	source
Burgers 2D	$u_t + u(u_x + u_y) = \nu \Delta u$ on $(0, 1)^2$, $u = 0$ on $\partial\Omega$	$u_0 = a \exp(- x - c ^2/2w^2)$, $c_i \sim U(0.15, 0.85)$, $w \sim U(0.05, 0.20)$, $a \sim U(0.5, 2)$; $\nu \sim$ $\log U(0.01, 0.1)$; outputs $t \in \{0.05, \dots, 0.25\}$	experiments/ mr-burgers2d/ engines.py: params_draw
Poisson 2D	$-\Delta u = f$ on $(0, 1)^2$, $u = 0$ on $\partial\Omega$	$f = a \exp(- x - c ^2/2w^2)$, $c_i \sim U(0.15, 0.85)$, $w \sim$ $\log U(0.02, 0.1)$, $a \sim U(0.5, 2)$	multistage- precision/ms_ parametric.py: sample_params
Poisson, L-shape	same source family on $(0, 1)^2 \setminus [\frac{1}{2}, 1)^2$	as above; sources centred in the removed quadrant rejected	experiments/ lshape
Heat 2D	$u_t = \kappa \Delta u$ on $(0, 1)^2$, $\kappa = 0.02$ fixed	polynomial-boundary Gaussian family (sin- gle_bc_poly_gaussian_v1); Crank–Nicolson $\Delta t = 0.025$	checkpoint_expanded_ seed790715 (lineage: linear-bank cell, job 3511417); printed CG measurements: job 3529772
Wave 2D (reflective)	$u_{tt} = c^2 \Delta u$ on $(0, 1)^2$, $u = 0$ on $\partial\Omega$	compact bump $\times$ Gaussian: half-widths $s_i \sim U(0.36, 0.42)$, centre $c_i \sim$ $U(s_i + 0.025, 1 - s_i - 0.025)$, amplitude $\sim U(0.7, 1.3)$, $\sigma_i \sim U(0.12, 0.16)$, advective velocity $v_i \sim$ $U(-0.5, 0.5)$ (zero every fourth case); speed $c \sim U(0.85, 1.15)$	experiments/ multiresolution- wave/audit_ dynamics.py: parameter_rows

head $k = 8$, fast = bank span $R' = 16$ (the cheapest span setting below 5 %); rows without this marker use correction-rank settings (q), pending the R' reruns; p timing did not pass its job's pre-registered neighbour/drift gates; speedups provisional (256^2 , 1024^2); a batched fit: all time steps of the linear autonomous heat problem solved together, same frozen model and settings; f held-out final cohort, evaluated once (all others development); t error over all output times including $t = 0$; e error over evolved output times only; c complete-query time with host transfers (all others GPU query); s stored quadrature rule, not confirmed on re-draws; x stored quadrature nodes with weights refit at this mesh, one held-out draw; ℓ 63×63 lattice quadrature rule, thin re-draw margin; w the same lattice rule at this mesh: five held-out draws pass, the confirmation draw does not; d dense residual. Burgers quadrature rules in detail: s is the stored rule of job 3780164 ($m = 2560$), which fails the confirmation draw at 256^2 ; x refits its weights at 512^2 ($m = 2438$); ℓ clears the re-draw bar by only 4×10^{-4} ($\rho_{\max} = 0.1156$ against 0.116). Over evolved times the accurate heat error is 0.21 % in two dimensions and 0.030 % in three. Poisson (dev. sources) rows use development sources of a separate run, not the held-out cohort. The FOM setting can differ between meshes of a series.

Table C.4 gives the settings and measured times behind every row of Table 1. Every cost and error come from the same invocation. Timings use GPU warm-up and synchronisation, double precision and the highest matrix-multiplication precision; all repetitions, including outliers, enter the median, and no time is borrowed from another allocation. Requested full fields and initialisation are charged; host transfer is included only where the table says complete query. For the held-out Burgers rows, an independent restricted-grid recomputation of the errors disagreed with the full-grid value by more than 5 % on 5/384 (2048^2) and 18/1152 (4096^2) case-arm rows; per-arm cohort worsts agree within 1.2 % and the full-grid recomputation is exact. For Burgers at 256^2 no quadrature rule passed the pre-registered re-draw procedure (its pick failed the confirmation draw), so Table 1 has no confirmed-rule row there; a post-hoc follow-up rule with an exact first time step passes all 12 draws of two jobs and gives 0.520 % at $0.085 \times$ against the same Newton–BiCGStab rule. For Burgers at 4096^2 , the lattice quadrature rule of the accurate rows passes five held-out draws but fails the independent confirmation draw ($\rho_{\max} = 0.1173$ against 0.116); from time step 2 on it passes ($\rho_{\max} = 0.0147$), so the failure sits in the first steps. Using the exact residual at the first step passes every check ($\rho_{\max} = 0.0513$) at the same error, but costs $49.6 \times$ as much and is then slower than the full-order solver ($0.10 \times$).

For Burgers at 4096^2 , copying the six double-precision output fields to the host adds about 246 ms to every arm. The Poisson and three-dimensional heat CG comparators are unpreconditioned; heat CG is warm-started from the previous time level.

Three-dimensional Poisson ($-\Delta u = f$, Gaussian sources) and heat ($u_t = \kappa \Delta u$, Gaussian initial fields) are posed on the unit cube with zero Dirichlet walls and extend the operators of Appendix A

Table C.4: Supporting data for Table 1, including the full-order setting the rule selected for each row: the two settings of each frozen model, median times (ms), timing scope and evidence status (allocations: Appendix C). “EQ” and “dense” name the Burgers residual evaluation; “single” marks models measured at one setting. Where a run recorded both timing scopes, the speedups in the scope not used by Table 1 are listed; a series never mixes scopes. The same column carries, for Burgers, the ratios against the tight Newton setting: that setting is far more accurate than the NM-ROM, so the rule never selects it and those ratios are context, not headline numbers. ^b this column’s time and its comparator were both measured in the later job named beside the FOM setting, so the ratio stays within one job. For heat that column gives the speedups against the tighter CN–CG setting (rtol 10^{-6}). The heat batched fit solves each output time independently against exactly propagated test moments, exploiting the linear, autonomous structure of the heat equation (eigenfunction tests). Rows from the paired-CG record list the one full-order setting that record retains, itself chosen by the same rule. * not in Table 1: a re-measurement of the row above it, a development-source run at a mesh whose held-out row is in Table 1, or the confirmed variant of the 4096² Burgers lattice rule with an exact residual at the first step. For the 4096² Burgers fast speedup in Table 1, the numerator is the time for Newton–BiCGStab, tol 10^{-3} , $\Delta t=0.01$, rather than the Newton time printed here.

Model	Mesh	Accuracy	Time	Accuracy	Time	FOM setting	FOM ms	Timing	Other scope, or the tighter one rule selects (FOM acc. / fast)	Status
Pressure 2D	256 ²	qm, $H^2 \leq 1.2$	qm, $H^2 \leq 1.2$	qm, $H^2 \leq 1.2$	qm, $H^2 \leq 1.2$	CN and 3x 10^{-4}	18.28	GPU query	7.05x / 7.72x (complete query)	development (timing previous ^a)
Pressure 2D	1024 ²	qm, $H^2 \leq 1.2$	qm, $H^2 \leq 1.2$	qm, $H^2 \leq 1.2$	qm, $H^2 \leq 1.2$	CN and 3x 10^{-4}	98.75	GPU query	14.8x / 22.3x (complete query)	development (timing previous ^a)
Pressure 2D	2048 ²	qm, $H^2 \leq 1.2$	qm, $H^2 \leq 1.2$	qm, $H^2 \leq 1.2$	qm, $H^2 \leq 1.2$	CN and 3x 10^{-4}	658.24	GPU query	26.0x / 78.2x (complete query)	development
Pressure 2D	4096 ²	qm, $H^2 \leq 1.2$	qm, $H^2 \leq 1.2$	qm, $H^2 \leq 1.2$	qm, $H^2 \leq 1.2$	CN and 3x 10^{-4}	4490.02	GPU query	46.3x / 188.2x (complete query)	development
Pressure, 4-stage 2D	256 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN, mid 10^{-4}	1.44	complete query	—	development
Pressure, 4-stage 2D	1024 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN, mid 10^{-4}	20.11	complete query	—	development
Pressure, 4-stage 2D	1024 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN, mid 10^{-4}	46.70	complete query	13.7x / 14.2x (GPU query)	development
Heat (wide bank, batched fit) 2D	256 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN, mid 10^{-4}	14.22	complete query	26.8x / 28.2x (GPU query)	development
Heat (wide bank, batched fit) 2D	1024 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN, mid 10^{-4}	45.51	GPU query	41.0x / 78.3x (vs. CN-CG and 10^{-6})	sealed hold-out
Heat (wide bank, batched fit) 2D	1024 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN-CG, $\Delta t=10^{-5}$, mid 10^{-4}	11.93	GPU query	426x / 448x (vs. CN-CG and 10^{-6})	sealed hold-out
Heat (wide bank, batched fit) 2D	4096 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN-CG, $\Delta t=10^{-5}$, mid 10^{-4}	52.92	GPU query	—	development
Pressure 2D	256 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	Newton-BiCGStab, tol 10^{-4}	31.79	GPU query	—	development
Pressure 2D	1024 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	Newton-BiCGStab, tol 10^{-4}	81.51	GPU query	—	development
Pressure 2D	2048 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	Newton-BiCGStab, tol 10^{-4}	122.76	GPU query	0.99x / 2.08x (complete query) 6.07x / 28.5x (vs. tight Newton)	development
Pressure 2D	4096 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	Newton-BiCGStab, tol 10^{-4}	523.85	GPU query	2.58x / 2.08x (complete query) 58.8x / 180.3x (vs. tight Newton)	development
Pressure, exact first step 2D ^b	4096 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	Newton-BiCGStab, tol 10^{-4}	523.76	GPU query	—	development
Pressure, exact first step 2D ^b	4096 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	Newton-BiCGStab, tol 10^{-4}	523.76	GPU query	—	development
Pressure 2D	256 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN, mid 10^{-4}	2.47	GPU query	—	accepted final
Pressure 2D	1024 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN, mid 10^{-4}	4.46	GPU query	—	accepted final
Pressure (late, accurate) 2D ^c	128 ³	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN, mid 10^{-4}	2.88	GPU query	1.87x / 1.36x (complete query)	development
Pressure (late, accurate) 2D ^c	128 ³	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN, mid 10^{-4}	12.85	GPU query	2.70x / 2.08x (complete query)	development
Pressure (late, accurate) 2D ^c	128 ³	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN, mid 10^{-4}	12.40	GPU query	2.68x / 2.27x (complete query)	development
Pressure (late, accurate) 2D ^c	256 ³	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN, mid 10^{-4}	16.87	GPU query	4.50x / 4.22x (complete query)	development
Heat (wide bank, batched fit) 2D	256 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN-CG, $\Delta t=10^{-5}$, mid 10^{-4}	3.31	GPU query	—	accepted final
Heat (wide bank, batched fit) 2D	1024 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN-CG, $\Delta t=10^{-5}$, mid 10^{-4}	5.23	GPU query	—	accepted final
Heat (wide bank, batched fit) 2D	1024 ²	$q \leq 0$	$q \leq 0$	$q \leq 0$	$q \leq 0$	CN-CG, $\Delta t=10^{-5}$, mid 10^{-4}	17.44	GPU query	—	accepted final
Newton-Newton 2D	256 ²	bank, $H^2 \leq 0$	bank, $H^2 \leq 0$	bank, $H^2 \leq 0$	bank, $H^2 \leq 0$	CNAB (spectral), 50 steps	6.84	GPU query	—	development
Newton-Newton 2D	1024 ²	bank, $H^2 \leq 0$	bank, $H^2 \leq 0$	bank, $H^2 \leq 0$	bank, $H^2 \leq 0$	CNAB (spectral), 50 steps	25.65	GPU query	—	development
Newton-Newton 2D	1024 ²	bank, $H^2 \leq 0$	bank, $H^2 \leq 0$	bank, $H^2 \leq 0$	bank, $H^2 \leq 0$	CNAB (spectral), 60 steps	108.43	GPU query	—	development

Table C.5: Heat at high resolution, accuracy: one frozen model per block; worst same-grid relative L^2 error over all output times and over evolved times, accurate / fast setting. CN: reduced Crank–Nicolson steps; batched fit: each output time fitted independently to exactly propagated test moments (linear autonomous problems with eigenfunction tests only). The sealed 2D cohort was opened once and gives every wide-bank row of Table 1; the development row ran on another GPU (A100) and is shown for reference. Wide bank: $R = 128$, $k = 8$.

Model	Mesh	Cohort	Stepping	q acc./fast	Err. all times (%)	Err. evolved (%)
Wide bank	1024 ²	development (12)	CN	32 / 0	0.24 / 0.56	0.17 / 0.46
	1024 ²	sealed (16)	CN	32 / 0	0.49 / 1.36	0.22 / 0.84
	1024 ²	sealed (16)	batched fit	32 / 0	0.49 / 1.36	0.21 / 0.84
	2048 ²	sealed (16)	CN	32 / 0	0.49 / 1.36	0.22 / 0.84
	2048 ²	sealed (16)	batched fit	32 / 0	0.49 / 1.36	0.21 / 0.84
	4096 ²	sealed (16)	CN	32 / 0	0.49 / 1.36	0.22 / 0.84
	4096 ²	sealed (16)	batched fit	32 / 0	0.49 / 1.36	0.21 / 0.84
Earlier ($R=32$)	2048 ²	development (12)	CN	24 / 0	1.68 / 4.56	1.11 / 4.56
	4096 ²	development (12)	CN	24 / 0	1.68 / 4.56	1.11 / 4.56
3D, earlier bank	128 ³	final (64)	CN	96 / 0	1.93 / 3.18	0.75 / 1.54
	128 ³	final (64)	batched fit	96 / 0	1.93 / 3.18	0.72 / 1.36
	256 ³	final, 1st 16 (16)	CN	96 / 0	1.27 / 2.02	0.55 / 0.91
	256 ³	final, 1st 16 (16)	batched fit	96 / 0	1.27 / 2.02	0.50 / 0.86

Table C.6: Heat at high resolution, timing for the rows of Table C.5: median GPU query times (ms), the FOM chosen by the rule of Table 1 and its speedup, and the speedup against CN–CG at $\Delta t = 0.025$, rtol 10^{-6} .

Model	Mesh	Cohort	Stepping	ms acc./fast	FOM ms	CN–CG FOM	Speedup	vs. 10^{-6}
Wide bank	1024 ²	development (12)	CN	31.2 / 36.9	66.8	$\Delta t=0.05$, rtol 10^{-3}	2.14 / 1.81x	6.33 / 5.35x
	1024 ²	sealed (16)	CN	27.3 / 27.8	43.5	$\Delta t=0.05$, rtol 10^{-3}	1.59 / 1.57x	4.70 / 4.62x
	1024 ²	sealed (16)	batched fit	3.1 / 3.4	43.5	$\Delta t=0.05$, rtol 10^{-3}	13.9 / 13.0x	41.0 / 38.3x
	2048 ²	sealed (16)	CN	28.0 / 28.4	272.5	$\Delta t=0.05$, rtol 10^{-3}	9.74 / 9.59x	28.8 / 28.3x
	2048 ²	sealed (16)	batched fit	4.5 / 4.8	272.5	$\Delta t=0.05$, rtol 10^{-3}	60.0 / 57.0x	177 / 169x
	4096 ²	sealed (16)	CN	31.7 / 32.2	1139.8	$\Delta t=0.05$, rtol 10^{-2}	36.0 / 35.4x	149 / 146x
	4096 ²	sealed (16)	batched fit	11.0 / 11.3	1139.8	$\Delta t=0.05$, rtol 10^{-2}	103 / 101x	428 / 416x
Earlier ($R=32$)	2048 ²	development (12)	CN	15.9 / 8.9	173.1	$\Delta t=0.1$, rtol 10^{-2}	10.9 / 19.5x	51.1 / 91.4x
	4096 ²	development (12)	CN	18.0 / 10.8	1014.6	$\Delta t=0.1$, rtol 10^{-2}	56.5 / 93.7x	267 / 442x
3D, earlier bank	128 ³	final (64)	CN	35.1 / 16.5	7.0	$\Delta t=0.1$, rtol 10^{-2}	0.20 / 0.43x	0.83 / 1.78x
	128 ³	final (64)	batched fit	5.2 / 4.0	7.0	$\Delta t=0.1$, rtol 10^{-2}	1.36 / 1.77x	5.67 / 7.40x
	256 ³	final, 1st 16 (16)	CN	37.0 / 21.0	173.3	$\Delta t=0.05$, rtol 10^{-4}	4.69 / 8.27x	8.96 / 15.8x
	256 ³	final, 1st 16 (16)	batched fit	9.6 / 8.4	173.3	$\Delta t=0.05$, rtol 10^{-4}	18.1 / 20.5x	34.7 / 39.3x

Table C.7: Supporting data for Table 2: every row of that table with its residual path and median GPU query time from the same allocation, plus the exploratory hyper-reduced Kim et al. rows, whose hyper-reduction did not pass its reproduction gate. Our rows use the unoptimised dense reference path (no empirical quadrature, stopping tolerance 10^{-6}); our optimised query is timed only in Table 1, in other jobs, and no ratio across jobs is formed. The shallow masked-autoencoder NM-LSPG of Kim et al. (2022) passed its reproduction gate, on their own 2D Burgers benchmark (their Sec. 6.2), on the third of three pre-registered attempts (sigmoid activation; median 1.45 % against a 1.5 % bar; published <1 %). Its published encoder width ($2n$) needs 136 GB to train at 256^2 and 2183 GB at 512^2 , beyond the 77 and 135 GB devices; at 512^2 its encoder, capped at width 4096, trained only 229–231 epochs in its 4800 s budget, so those errors partly reflect a training-time limit. The data-matched row is trained on our bank’s 576 trajectories.

Mesh	Method	Residual path	k	Worst (%)	Median (%)	GPU ms (same job)	MB
256^2	Kim et al. NM-LSPG	dense	8	163.93	43.34	1301	2480
256^2	Kim et al. NM-LSPG	dense	16	145.40	43.87	2384	2567
256^2	Kim et al. NM-LSPG	dense	32	127.51	43.61	3846	2742
256^2	Kim et al. NM-LSPG, data-matched	dense	16	120.33	38.71	3886	2567
256^2	POD-LSPG	dense	8	55.52	22.72	138	18
256^2	POD-LSPG	dense	16	41.53	10.45	259	30
256^2	POD-LSPG	dense	32	24.78	5.65	519	55
256^2	This work, fast ($q = 0, k = 16$)	dense (reference path)	16	6.79	0.66	262	625
256^2	This work, accurate ($q = 256, k = 16$)	dense (reference path)	272	0.88	0.089	3746	1769
256^2	Kim et al. NM-LSPG-HR (exploratory)	hyper-reduced, not reproduced	8	115.60	51.78	65	2452
256^2	Kim et al. NM-LSPG-HR (exploratory)	hyper-reduced, not reproduced	16	202.87	52.51	126	2506
256^2	Kim et al. NM-LSPG-HR (exploratory)	hyper-reduced, not reproduced	32	122.68	41.98	475	2794
512^2	Kim et al. NM-LSPG	dense	8	175.71	55.48	746	9959
512^2	Kim et al. NM-LSPG	dense	16	174.99	58.56	1497	10511
512^2	Kim et al. NM-LSPG	dense	32	252.33	56.72	2059	10879
512^2	POD-LSPG	dense	8	55.82	24.08	314	71
512^2	POD-LSPG	dense	16	42.01	13.83	617	122
512^2	POD-LSPG	dense	32	25.63	7.74	383	222
512^2	This work, fast ($q = 0, k = 16$)	dense (reference path)	16	7.72	0.69	404	2368
512^2	This work, accurate ($q = 256, k = 16$)	dense (reference path)	272	1.05	0.080	6269	6891
512^2	Kim et al. NM-LSPG-HR (exploratory)	hyper-reduced, not reproduced	8	273.27	53.08	30	9813
512^2	Kim et al. NM-LSPG-HR (exploratory)	hyper-reduced, not reproduced	16	185.17	58.52	62	10099
512^2	Kim et al. NM-LSPG-HR (exploratory)	hyper-reduced, not reproduced	32	194.47	56.00	138	10569

Table C.8: Dense and empirical-quadrature (EQ) residual evaluation on Burgers 2D at 256^2 . Error is worst evolved relative L^2 (%); time is median GPU milliseconds from one allocation (job 3780164, the EQ-ladder job of Table D.1; the 256^2 row of Table 1 is job 3789570, with its own measured error); the last column is dense time over EQ time. Dashes: no paired dense measurement. Rules at $q = 0, 16, 32$ pass every independent reconstruction; higher-rank rules pass some (Table D.1).

Correction rank q	Dense error (%)	Dense ms	EQ error (%)	EQ ms	Dense / EQ
0	1.8890	292.87	1.8891	59.07	4.96×
16	—	—	1.4270	80.61	—
32	—	—	1.2493	97.70	—
64	1.0843	624.93	1.2275	120.63	5.18×
128	0.8930	1190.49	0.8936	246.87	4.82×
256	0.5194	3915.14	0.5389	722.21	5.42×

Table C.9: Three-dimensional configurations, read from the run records. Domains are the unit cube with homogeneous Dirichlet data, except Navier–Stokes, which is periodic.

Problem	Mesh	Cases	Reduced model	Named FOM	Residual
Poisson 3D	$32^3, 64^3$	64 final	$q \in \{0, 32, 96\}$ ($k = 16$)	CG, $\text{rot } 10^{-2}$, no preconditioner	dense
Heat 3D (new bank)	$32^3, 64^3, 128^3$	64 sealed final (all times)	$q \in \{0, 288\}$ ($k = 32, R = 320$)	CN-CG, setting per row by the rule; named $\Delta t = 0.025$, $\text{rot } 10^{-6}$	exact (linear)
Navier–Stokes 3D	$32^3, 64^3, 96^3$ periodic	16 dev. + 32 held-out (96^3)	POD bank $R = 64$ in a moving frame; head $k = 8$; span $R' \in \{64, 48, 32, 16, 8\}$	CNAB2 (spectral), 50–70 steps (per row by the rule)	exact (precomputed quadratic tensor)

across three axes. Navier–Stokes is described in Appendix A.4; its FOM is the pseudo-spectral CNAB2 solver, not a CG solve.

C.2 SOURCE RECORDS

The accompanying source package retains the full experiment archive, including failed configurations, retractions, training records, checkpoints, case-level fields, timing repetitions and independent audits. The PDF contains the evidence needed for its stated claims rather than every experiment in that archive. Table generators and frozen evidence identify the displayed values through these machine-readable manifests:

- `tables/provenance.json`: original two-dimensional studies.
- `tables/rewrite-provenance.json`: paired CG comparisons.
- `tables/burgers-iterative-provenance.json`: the earlier fine-grid Burgers tight and relaxed controls.
- `tables/headline-provenance.json`: every row of Tables 1, C.4 and C.9 and every row of Table 3, with the commit and SHA256 of each source record.

Each manifest records source hashes; its corresponding evidence retains solver configuration, checkpoint identity and allocation metadata. Development selection and final evaluation are distinct. Solver exit flags document numerical stopping, not a proof of global optimality.

Allocations. Every ratio pairs times from one Slurm allocation. Tables 1 and C.4: Poisson 2D 4199318, 4199321; Poisson, L-shape 2D 3784663, 3789568, 4057694; Heat (wide bank, batched fit) 2D 4060928; Burgers 2D 3789570, 3805065, 3789572, 4071616, 4079320; Burgers, exact first step 2D 4153483; Poisson 3D 4028642; Poisson (dev. sources) 3D 4051032, 4056288; Heat (new bank, batched fit) 3D 4153878; Navier–Stokes 3D 4198840, 4198090, 4198101. Table C.9: Poisson 3D 4028642; Heat 3D 4153878; Navier–Stokes 3D 4198840, 4198090, 4198101, 4202872. Table C.5: 4051290, 4054869, 4060928, 4072943. Tables 2 and C.7: 256^2 4095408, 512^2 4107272 (reproduction gate: 4077574).

Training/evaluation separation (Burgers). The 4608 training trajectories of the Burgers checkpoint were regenerated from their seeds and compared with every evaluation cohort (the 32 validation cases of Table 2, the 64 held-out cases, the 6 development cases, the sealed cohort): no cohort shares a parameter vector, or any single parameter value, with the training set (check script and output in `evidence/burgers-train-eval-overlap-2026-09-21`).

D QUADRATURE AND SOLVER CONTROLS

Table D.1: The EQ ladder with the cheapest rule passing the primary bar in its draw per rung, timed in one allocation (job 3780164, A100 80 GB), with its same-job dense twins, and the construction status from the four-draw replication (job 3783811): confirmed = every re-draw passes, marginal = only the listed fraction does. Errors are as measured with these rules (SHA256-identified); they are not properties of the construction above $q = 32$. Ladder rows are read from the study’s final summary record.

q	m	fit states	$\rho_{\max}$ (this draw)	construction status	EQ evolved %	EQ all %	EQ ms	dense evolved %	dense ms	EQ/dense cost
0	1024	64	0.0153	confirmed (3 of 3 re-draws)	1.8891	2.5629	59.1	1.8890	292.9	0.202
16	1024	64	0.0935	confirmed (3 of 3 re-draws)	1.4270	2.4806	80.6	—	—	—
32	1024	42	0.0533	confirmed (2 of 2 re-draws)	1.2493	2.3534	97.7	—	—	—
64	1024	25	0.0531	marginal (2 of 6 draws pass)	1.2275	2.1489	120.6	1.0843	624.9	0.193
128	2048	64	0.0669	marginal (4 of 5 draws pass)	0.8936	1.8116	246.9	0.8930	1190.5	0.207
256	2048	64	0.1074	marginal (1 of 5 draws pass)	0.5389	0.9053	722.2	0.5194	3915.1	0.184

Reading these tables. M is the number of weak test modes and m the quadrature node count. A rule is confirmed when every independent reconstruction of it passes the bar, and marginal when only some do; these statuses concern the rule construction, not the error of one trajectory. Same-grid error and error against a refined reference measure different quantities.

Table D.2: Solver-side controls on Burgers 2D, 32 validation cases, one allocation: stopping tolerance, EQ node count m and iteration cap, with full-order settings for scale. Error: worst over cases and output times of the L^2 error against the refined reference, relative to the initial-field norm, so the full-order rows show their discretisation error.

setting	worst % (32 validation)	device ms	early-stopped
EQ $m=256$, tol 10^{-6} , cap 180	7.245	51.0	0/96
EQ $m=512$, tol 10^{-8}	6.712	64.7	0/96
EQ $m=512$, tol 10^{-3}	6.701	41.6	0/96
EQ $m=512$, cap 2 (early-stopped)	90.324	28.4	96/96
FOM Newton 10^{-2} , $\Delta t=0.01$	6.807	9.9	0/96
FOM Newton 10^{-2} , $\Delta t=0.005$	35.357	17.4	0/96
FOM Newton 10^{-4} , $\Delta t=0.005$	6.171	22.4	0/96
FOM Newton 10^{-6} , $\Delta t=0.005$	6.172	88.3	0/96